Copyright All rights reserved

7. 내적

벡터 공간의 분석과 응용

Heesung Oh
Copyright All rights reserved

- **7.1.1 내적의 성질**

$$\vec{A} \cdot \vec{B} = \sum A_k B_k$$

- **내적과 벡터의 크기**

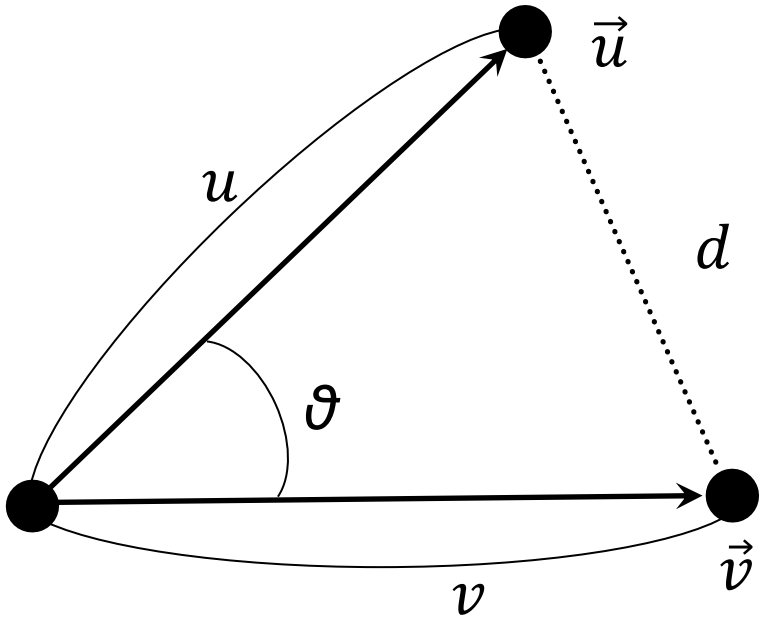
$$\vec{v} \cdot \vec{v} = |\vec{v}|^2$$

- **합성 벡터의 내적**

$$\begin{aligned}(\vec{u} + \vec{v}) \cdot (\vec{u} + \vec{v}) &= \vec{u} \cdot \vec{u} + \vec{v} \cdot \vec{v} + 2(\vec{u} \cdot \vec{v}) \\ &= |\vec{u}|^2 + |\vec{v}|^2 + 2(\vec{u} \cdot \vec{v})\end{aligned}$$

Heesung Oh
Copyright All rights reserved

- **내적 유도**



$$d^2 = u^2 + v^2 - 2uv\cos\theta$$

$$(\vec{u} - \vec{v}) \cdot (\vec{u} - \vec{v})$$
$$= \vec{u} \cdot \vec{u} + \vec{v} \cdot \vec{v} - 2(\vec{u} \cdot \vec{v})$$
$$= |\vec{u}|^2 + |\vec{v}|^2 - 2(\vec{u} \cdot \vec{v})$$

$$d^2 = |(\vec{u} - \vec{v})|^2$$
$$= (\vec{u} - \vec{v}) \cdot (\vec{u} - \vec{v})$$
$$= |\vec{u}|^2 + |\vec{v}|^2 - 2(\vec{u} \cdot \vec{v})$$
$$= u^2 + v^2 - 2(\vec{u} \cdot \vec{v}) = u^2 + v^2 - 2uv\cos\theta$$

$$-2(\vec{u} \cdot \vec{v}) = -2uv\cos\theta$$

$$\vec{u} \cdot \vec{v} = |\vec{u}||\vec{v}|cos\theta$$

7.1.3 행렬의 곱셈을 내적으로 표현

Heesung Oh
Copyright All rights reserved

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}$$

$$\begin{pmatrix} ax + by \\ cx + dy \end{pmatrix} = \begin{pmatrix} (a, b) \cdot (x, y) \\ (c, d) \cdot (x, y) \end{pmatrix}$$

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}$$

$$= \begin{pmatrix} (a, b) \cdot (e, g) & (a, b) \cdot (f, h) \\ (c, d) \cdot (e, g) & (c, d) \cdot (f, h) \end{pmatrix}$$

Heesung Oh
Copyright All rights reserved

- 직교 행렬(Orthogonal Matrix) 특징
  - Ex) 회전 행렬

$$M^{-1} = M^t$$

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{t}$$

$$= \begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} a & c \\ b & d \end{pmatrix} = \begin{pmatrix} (a,b)\cdot(a,b) & (a,b)\cdot(c,d) \\ (a,b)\cdot(c,d) & (c,d)\cdot(c,d) \end{pmatrix}$$

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$R_\theta R_\theta^t = \begin{pmatrix} \cos^2\theta + \sin^2\theta & 0 \\ 0 & \sin^2\theta + \cos^2\theta \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

7.1.3 행렬의 곱셈을 내적으로 표현

Heesung Oh
Copyright All rights reserved

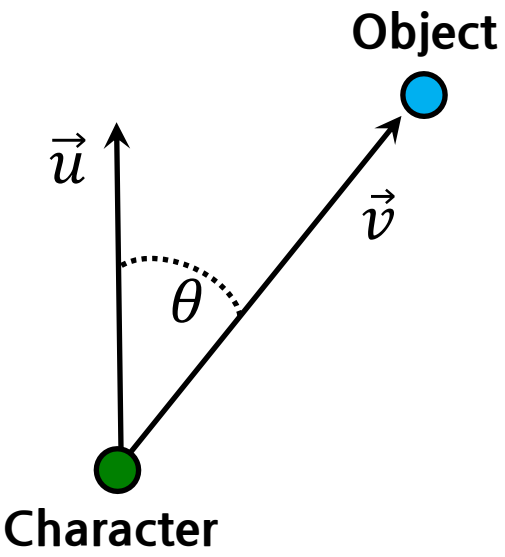
- 강체 변환 (Rigid Transformation) 특징
  - 변환된 기저벡터의 크기는 1
  - 모든 기저 벡터는 서로 직교
  - 행렬식 값 = 1

  - Ex) 회전 벡터의 행렬식 $\det(R) = \cos^2\theta + \sin^2\theta = 1$

7.2.1 앞뒤 판별

Heesung Oh
Copyright All rights reserved

- 앞뒤 판별
  - 내적 결과 양수
    - 두 벡터는 같은 방향
  - 내적 결과 음수
    - 두 벡터는 다른 방향

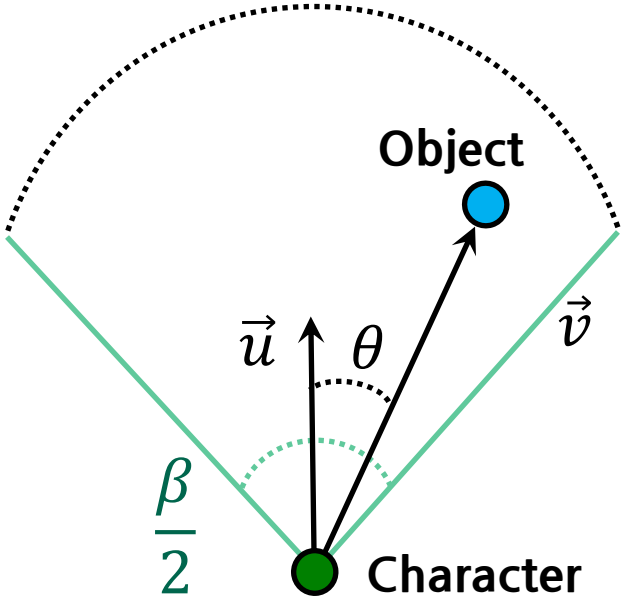


- 시선 벡터: $\vec{u}$, Object 벡터 $\vec{v}$
  - $\vec{u} \cdot \vec{v}$ 결과
    - 양수: 캐릭터 앞에 목표물
    - 음수: 캐릭터 뒤에 목표물
    - 0: 캐릭터 바로 옆(좌/우)

7.2.2 시야 판별

Heesung Oh
Copyright All rights reserved

- 시야 각 $\cos\left(\frac{\beta}{2}\right)$ 미리 계산
- $\vec{u}$, $\vec{v}$ Normalize
- 목표물이 시야 안
  - $\hat{u} \cdot \hat{v} \geq \cos\left(\frac{\beta}{2}\right)$
- 목표물이 시야 밖
  - $\hat{u} \cdot \hat{v} < \cos\left(\frac{\beta}{2}\right)$



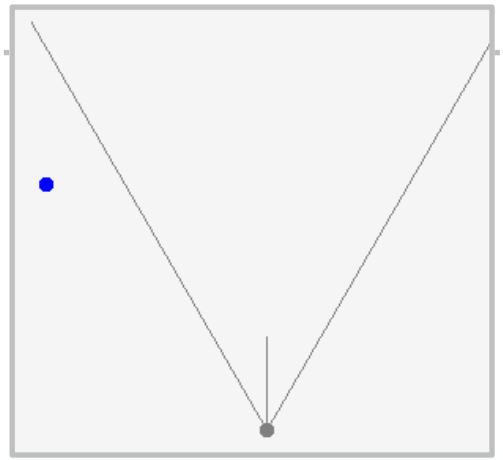
7-1 내적을 활용한 목표물 감지

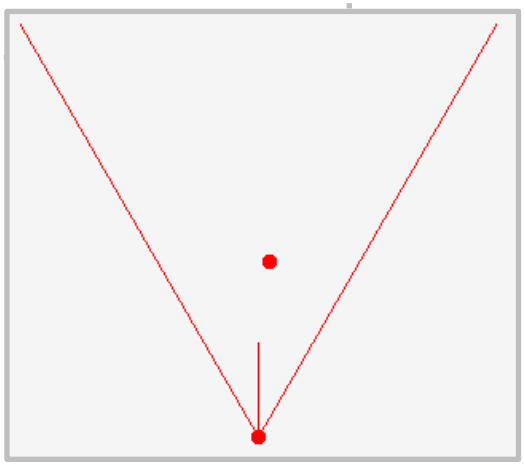
Heesung Oh
Copyright All rights reserved

```
// 시야각의 코사인 값은 최초 1회만 계산해 보관한다.
float fovAngle = 60.f;
static float halfFovCos = cosf(Math::Deg2Rad(fovAngle * 0.5f));
```

```
Vector2 f = Vector2::UnitY;
Vector2 v = (targetPosition - playerPosition).GetNormalize();

// 물체의 최종 상태 설정
if (v.Dot(f) >= halfFovCos)
{
  playerColor = LinearColor::Red;
  targetColor = LinearColor::Red;
}
else
{
  playerColor = LinearColor::Gray;
  targetColor = LinearColor::Blue;
}
```



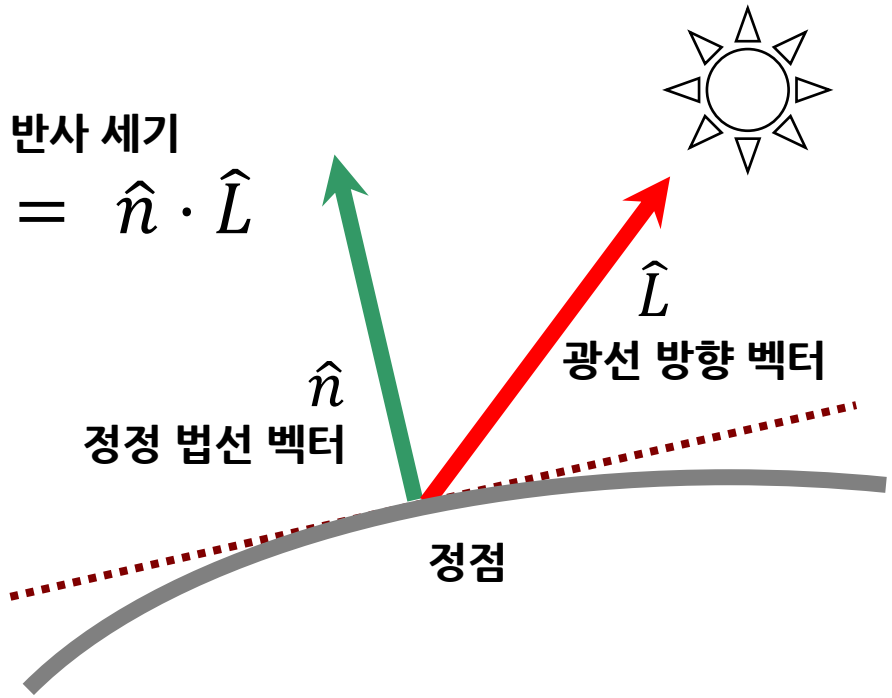


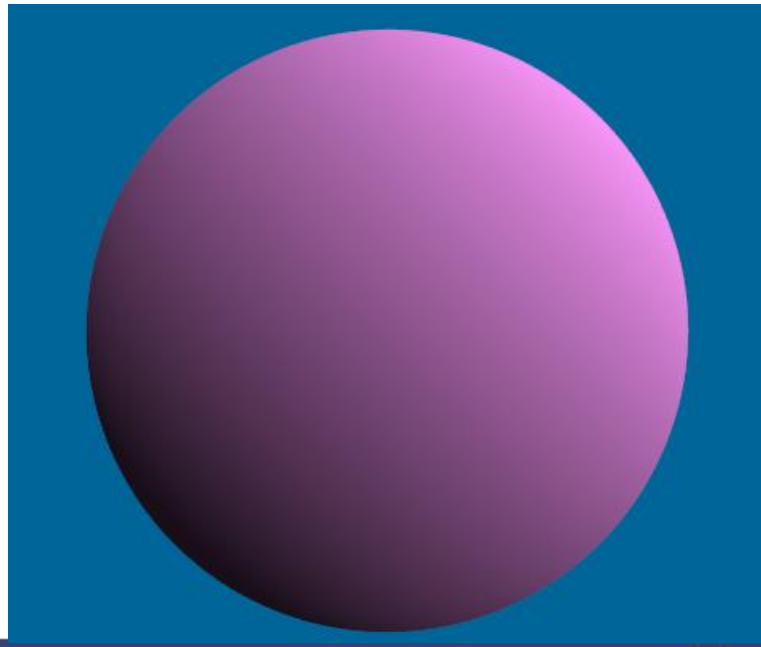
Heesung Oh
Copyright All rights reserved

- 램버트 반사(Lambertian Reflection) 모델
  - 반사 세기 = Dot(정점의 법선(N), 빛의 방향(L))
  - 내적의 범위가 [-1, 1] 이므로 반사 세기 범위를 [0,1]로 변경
  - 반사 세기 = (1+ Dot(N, L))/2

반사 세기

$$I = \hat{n} \cdot \hat{L}$$

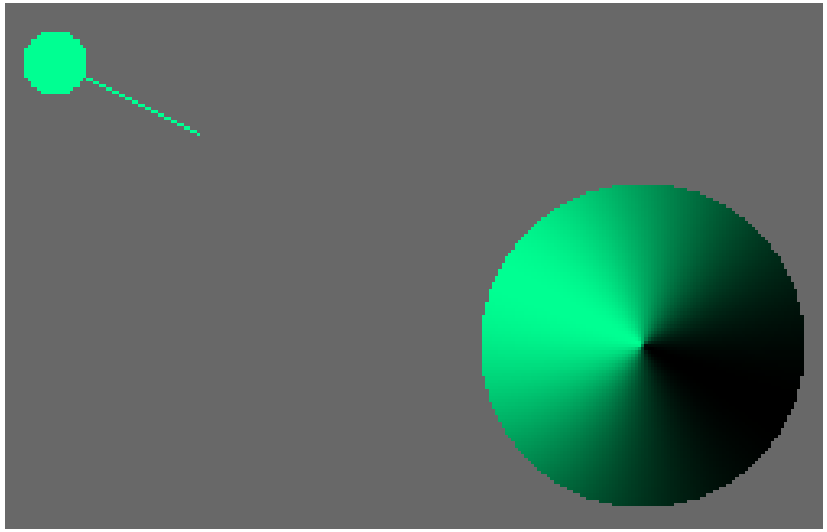




7-2 내적을 활용한 조명 모델 구현

Heesung Oh
Copyright All rights reserved

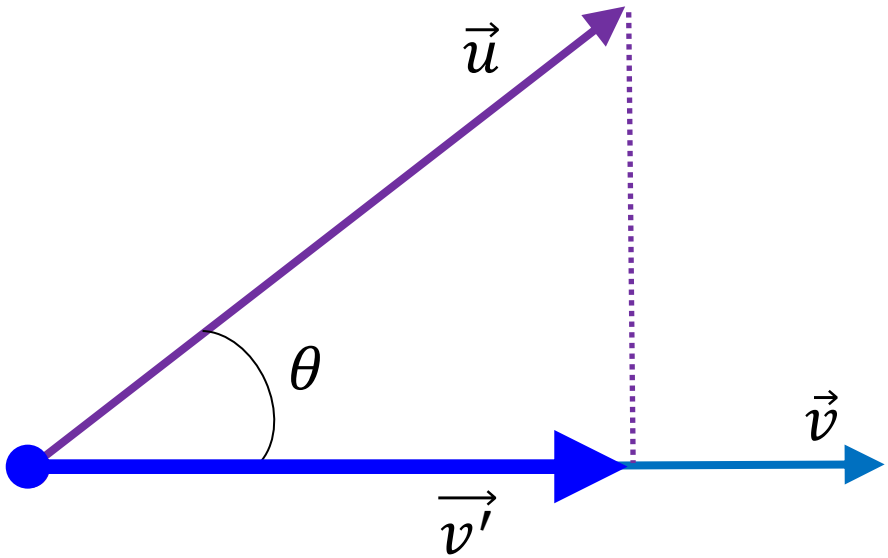
```
Vector2 n = (v - circlePosition).GetNormalize();
Vector2 l = (lightPosition - v).GetNormalize();
float shading = Math::Clamp(n.Dot(l), 0.f, 1.f);
```



7.4 투영 벡터

Heesung Oh
Copyright All rights reserved

$$\overrightarrow{v'} = |\vec{u}|\cos\theta\,\hat{v}$$
$$= |\vec{u}|\left(\frac{\vec{u}\cdot\vec{v}}{|\vec{u}||\vec{v}|}\right)\hat{v}$$
$$= |\vec{u}|\frac{\vec{u}\cdot\vec{v}}{|\vec{u}||\vec{v}|}\hat{v}$$
$$= |\vec{u}|\frac{\vec{u}\cdot\vec{v}}{|\vec{u}||\vec{v}|}\frac{\vec{v}}{|\vec{v}|}$$
$$= \frac{\vec{u}\cdot\vec{v}}{|\vec{v}||\vec{v}|}\vec{v} = \frac{\vec{u}\cdot\vec{v}}{|\vec{v}|^2}\vec{v}$$



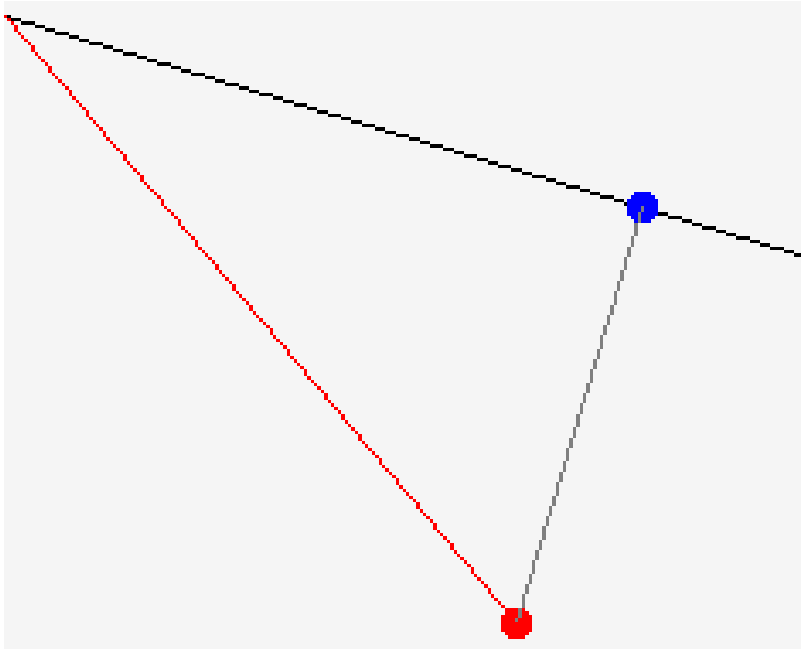
$$\overrightarrow{v'} = \frac{\vec{u}\cdot\vec{v}}{\vec{v}\cdot\vec{v}}\vec{v}$$

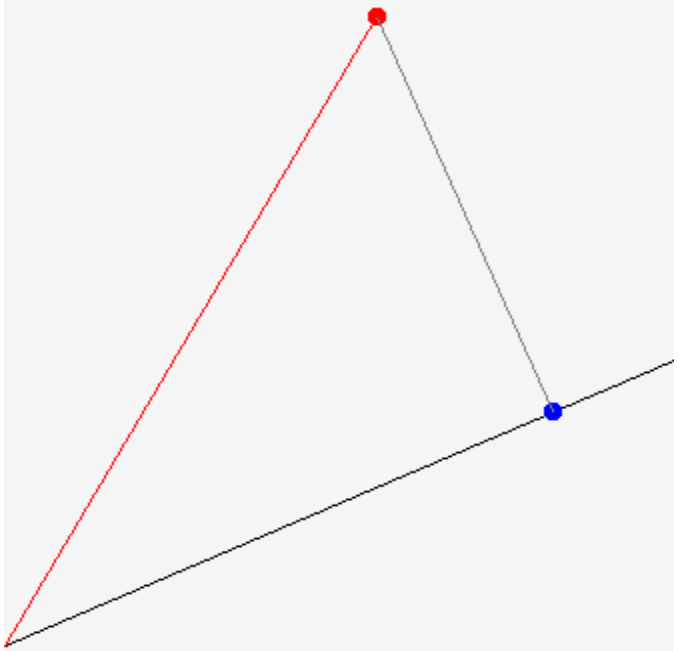
$$\overrightarrow{v'} = (\vec{u}\cdot\vec{v})\vec{v} \quad , \ |\vec{v}| = 1$$

7-3 내적을 활용한 벡터의 투영

Heesung Oh
Copyright All rights reserved

```
// 투영된 위치와 거리 계산
Vector2 unitV = (lineEnd - lineStart).GetNormalize();
Vector2 u = point - lineStart;
Vector2 projV = unitV * (u.Dot(unitV));
Vector2 projectedPoint = lineStart + projV;
float distance = (projectedPoint - point).Size();
```





7.5 정리

Heesung Oh
Copyright All rights reserved

- 벡터 내적
  - 행렬 곱셈
  - 앞/뒤 판정
  - 시야 판정
  - 램버트 반사
  - 투영 벡터

Copyright All rights reserved

8. 삼각형

물체를 구성하는 가장 작은 단위

8.1 세 점의 결합

Heesung Oh
Copyright All rights reserved

- **세 점**
  - 세 점 중 하나는 나머지 독립된 두점으로 표현 가능

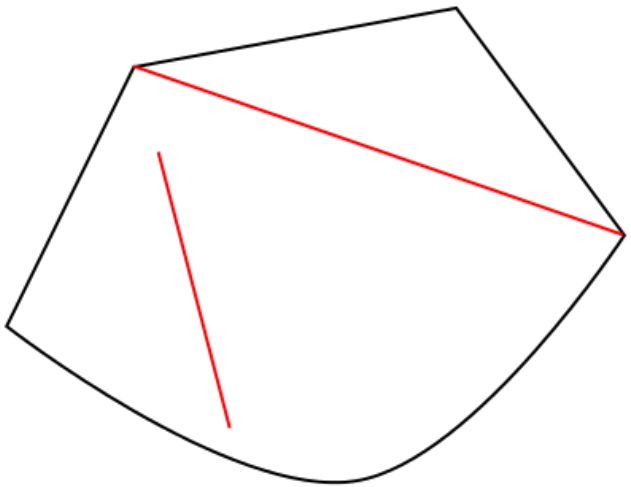
$$P' = s \cdot P_1 + s \cdot P_2 + (1 - s - t) \cdot P_3$$
$$P' - P_3 = s(P_1 - P_3) + t(P_2 - P_3)$$
$$\vec{w} = s\vec{u} + t\vec{v}$$

- **컨벡스 결합 (Convex Combination)**

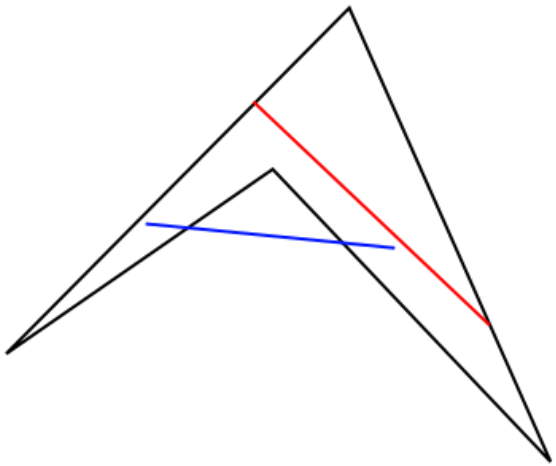
$$\frac{\sum c_i \cdot P_i}{\sum c_i} \quad , \text{ s.t. } 0 \leq c_i \leq 1$$

Heesung Oh
Copyright All rights reserved

- 컨벡스 영역 (Convex region)
  - 영역 내 임의의 두 점에서 두 점을 잇는 선분이 항상 내부에 있으면 Convex(볼록)



Convex



Not - convex

8.1 세점의 결합

Heesung Oh
Copyright All rights reserved

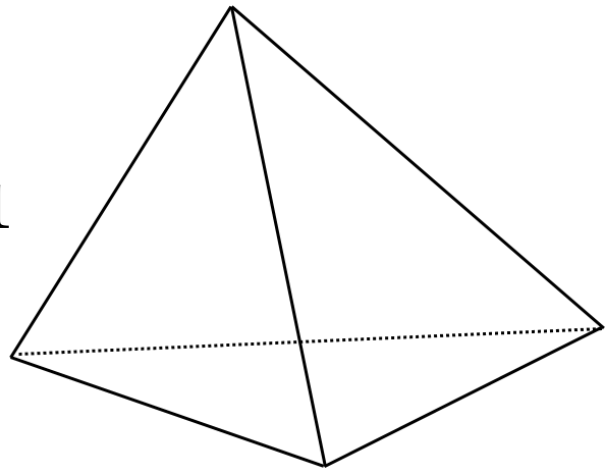
- 3점의 아핀 결합

$$P = \frac{c_1 P_1 + c_2 P_2 + c_3 P_3}{c_1 + c_2 + c_3} \qquad 0 \le c_i \le 1$$

  - Convex: 두 점을 이은 선분이 항상 내부에 존재

- 삼각뿔(Tetrahedron)
  - Tetrahedron = 4점의 convex 조합으로 생성된 3D 볼록 영역

$$P = \frac{c_1 P_1 + c_2 P_2 + c_3 P_3 + c_4 P_4}{c_1 + c_2 + c_3 + c_4} \qquad 0 \le c_i \le 1$$



Heesung Oh
Copyright All rights reserved

메시

- 3D 공간에서
  물체의 외형을 표현하기 위해
  정점(Vertex), 간선(Edge), 면(Face)을 결합한
  기하학적 데이터 구조

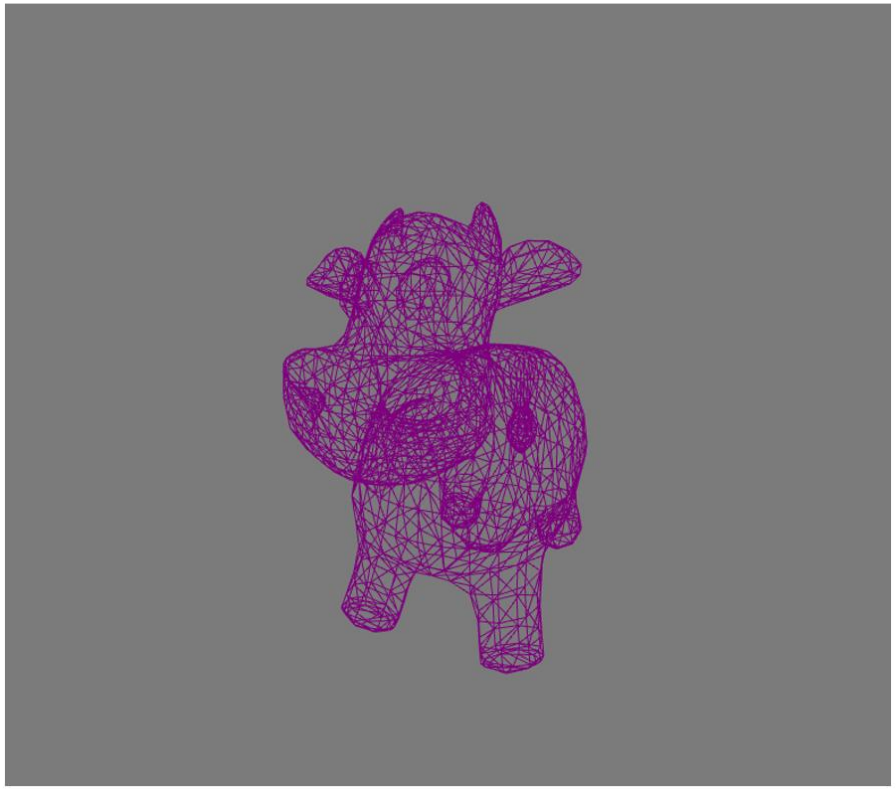
- 정점 버퍼(Vertex Buffer)
  - 3D 공간 상의 좌표, 색상, UV 등
    각 정점의 상세 속성 데이터를 저장하는 메모리 블록

- 인덱스 버퍼(Index Buffer)
  - 정점 버퍼의 번호(인덱스)를 참조하여
    점들을 연결하는 순서를 정의, 삼각형 면을 구성하는 설계도
  - 인덱스 버퍼 개수 = 삼각형의 개수 X 3

8.2 메시

Heesung Oh
Copyright All rights reserved

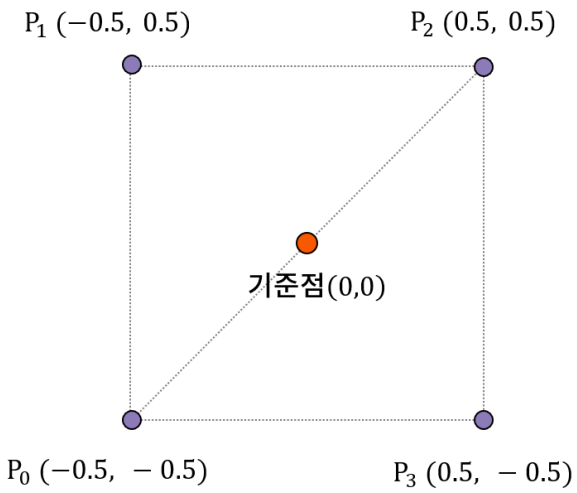
(a)



(b)

8-1 와이어프레임 렌더링의 구현

Heesung Oh
Copyright All rights reserved

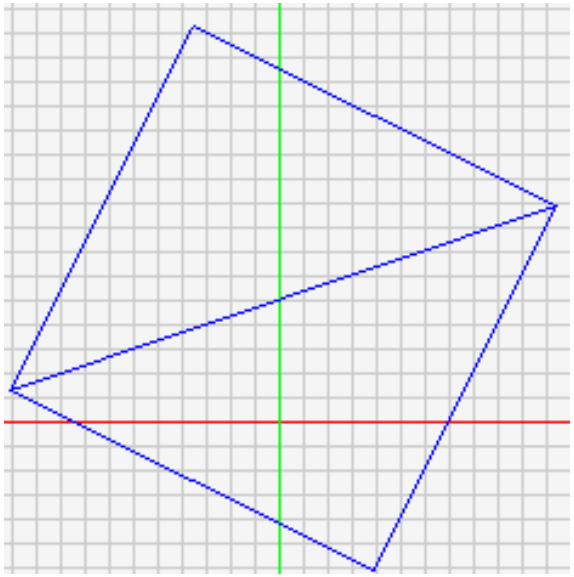


정점 버퍼

순서	좌표
0	(−0.5, − 0.5)
1	(−0.5, 0.5)
2	(0.5, 0.5)
3	(0.5, − 0.5)

인덱스 버퍼

순서	삼각형	정점 순서
0	0	0
1	0	1
2	0	2
3	1	0
4	1	2
5	1	3



```
std::array<Vertex2D, vertexCount> rawVertices = {
  Vertex2D(Vector2(-squareHalfSize, -squareHalfSize)),
  Vertex2D(Vector2(-squareHalfSize, squareHalfSize)),
  Vertex2D(Vector2(squareHalfSize, squareHalfSize)),
  Vertex2D(Vector2(squareHalfSize, -squareHalfSize))
};
```

```
std::array<size_t, triangleCount * 3> indices = {
    0, 1, 2,
    0, 2, 3
};
```

Heesung Oh
Copyright All rights reserved

- **3점의 아핀 결합**

$$X = sP_1 + tP_2 + (1 - s - t)P_3$$
$$X = s(P_1 - P_3) + t(P_2 - P_3) + P_3$$
$$X - P_3 = s(P_1 - P_3) + t(P_2 - P_3)$$

$$\vec{x} = X - P_3$$
$$\vec{u} = P_1 - P_3$$
$$\vec{v} = P_2 - P_3$$

$$\Rightarrow \quad \vec{x} = s\vec{u} + t\vec{v}$$

$$s = \frac{\begin{vmatrix} \vec{u}\cdot\vec{x} & \vec{u}\cdot\vec{v} \\ \vec{v}\cdot\vec{x} & \vec{v}\cdot\vec{v} \end{vmatrix}}{\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{v} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{v} \end{vmatrix}} \qquad t = \frac{\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{x} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{x} \end{vmatrix}}{\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{v} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{v} \end{vmatrix}}$$

$$\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{v} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{v} \end{vmatrix} = (\vec{u}\cdot\vec{u})(\vec{v}\cdot\vec{v}) - (\vec{u}\cdot\vec{v})(\vec{u}\cdot\vec{v}) = |\vec{u}|^2|\vec{v}|^2 - (\vec{u}\cdot\vec{v})^2$$

8.3.1 무게중심좌표의 계산

Heesung Oh
Copyright All rights reserved

$$s = \frac{\begin{vmatrix} \vec{u}\cdot\vec{x} & \vec{u}\cdot\vec{v} \\ \vec{v}\cdot\vec{x} & \vec{v}\cdot\vec{v} \end{vmatrix}}{\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{v} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{v} \end{vmatrix}} \quad t = \frac{\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{x} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{x} \end{vmatrix}}{\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{v} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{v} \end{vmatrix}}$$

▷ 삼각형 안에 위치할 조건

$$0 \le s \le 1 \quad 0 \le t \le 1 \quad 0 \le s + t \le 1$$

▷ 퇴화 삼각형

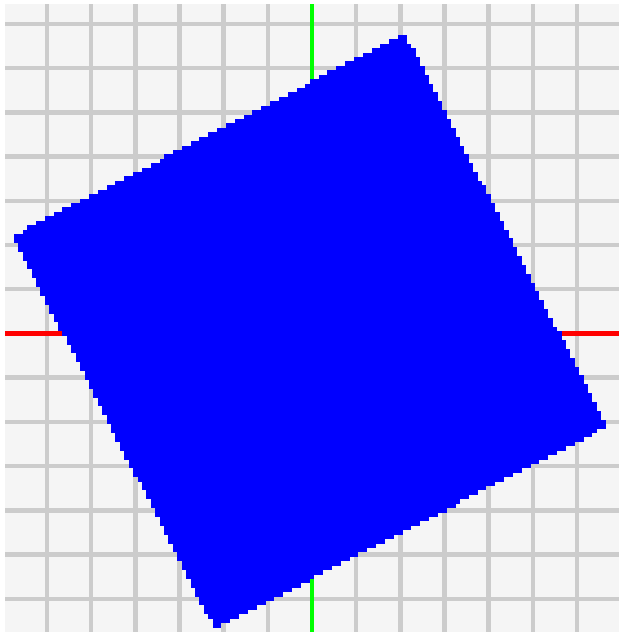
$$\begin{vmatrix} \vec{u}\cdot\vec{u} & \vec{u}\cdot\vec{v} \\ \vec{u}\cdot\vec{v} & \vec{v}\cdot\vec{v} \end{vmatrix} = 0 \Rightarrow |\vec{u}|^2|\vec{v}|^2 - (\vec{u}\cdot\vec{v})^2 = 0$$

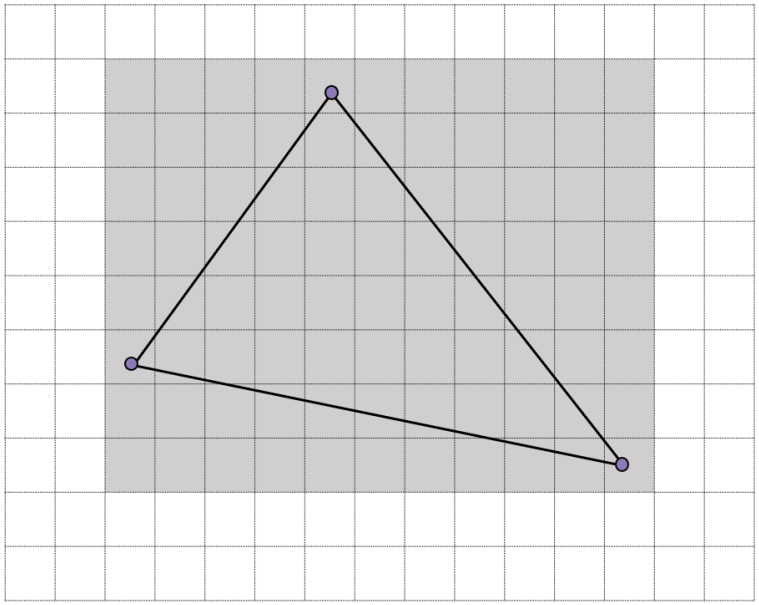
$$(1 - cos^2\theta) = 0$$

8-2 삼각형 칠하기

Heesung Oh
Copyright All rights reserved

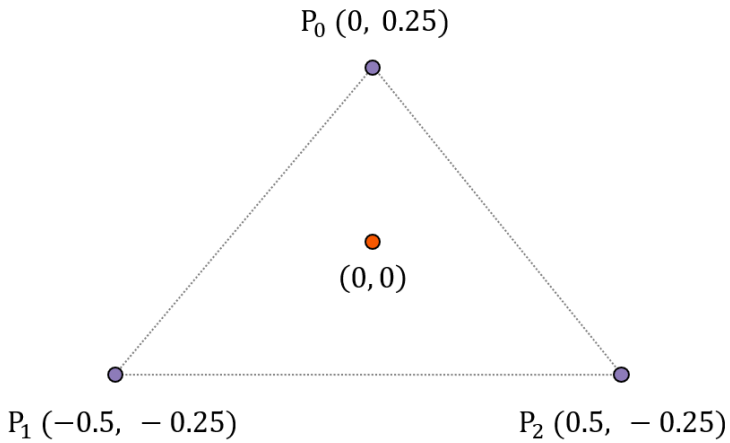
```
ScreenPoint fragment = ScreenPoint(x, y);
Vector2 pointToTest = fragment.ToCartesianCoordinate(_ScreenSize);
Vector2 w = pointToTest - tv[0].Position;
float wdotu = w.Dot(u);
float wdotv = w.Dot(v);
// 분자 값을 구하고 최종 무게중심좌표 산출
float s = (wdotv * udotv - wdotu * vdotv) * invDenominator;
float t = (wdotu * udotv - wdotv * udotu) * invDenominator;
float oneMinusST = 1.f - s - t;
// 컨벡스 조건을 만족할 때만 점 찍기
if (((s >= 0.f) && (s <= 1.f)) && ((t >= 0.f) && (t <= 1.f)) &&
   ((oneMinusST >= 0.f) && (oneMinusST <= 1.f)))
{
    r.DrawPoint(fragment, LinearColor::Blue);
}
```





8-3 무지개 삼각형 만들기

Heesung Oh
Copyright All rights reserved

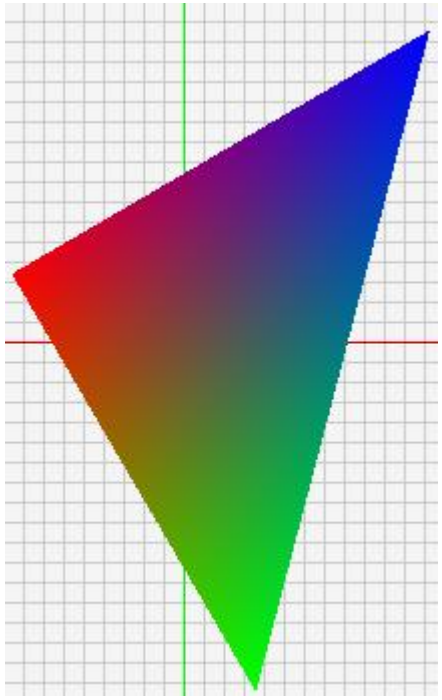


정점 버퍼

인덱스	좌표	색상(RGB)
0	(0, 0.25)	(1, 0, 0)
1	(−0.5, − 0.25)	(0, 1, 0)
2	(0.5, − 0.25)	(0, 0, 1)

인덱스 버퍼

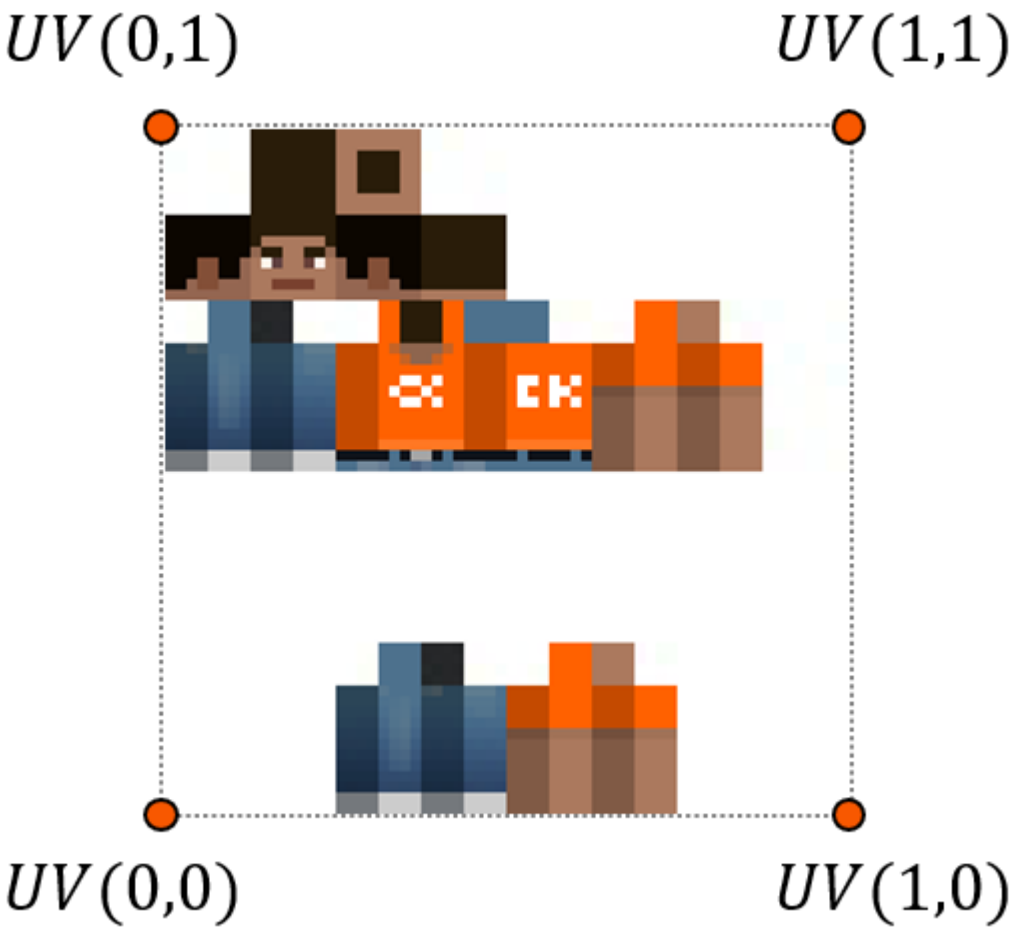
인덱스	삼각형	정점인덱스
0	0	0
1	0	2
2	0	1



```
if (((s >= 0.f) && (s <= 1.f)) &&
    ((t >= 0.f) && (t <= 1.f)) &&
    ((oneMinusST >= 0.f) && (oneMinusST <= 1.f)))
{
  LinearColor outColor = tv[0].Color * oneMinusST
                       + tv[1].Color * s + tv[2].Color * t;
  r.DrawPoint(fragment, outColor);
```

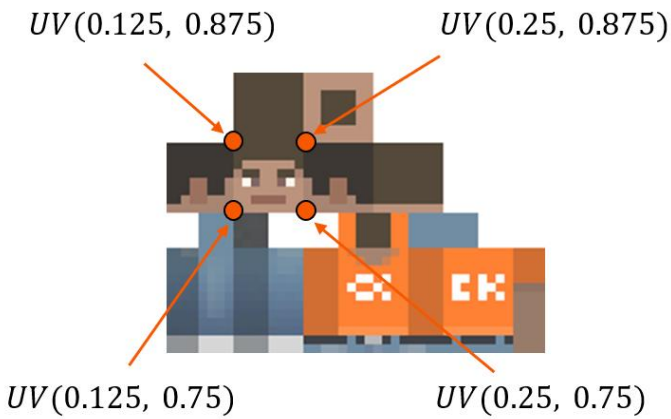
8.4 텍스처 매핑

Heesung Oh
Copyright All rights reserved

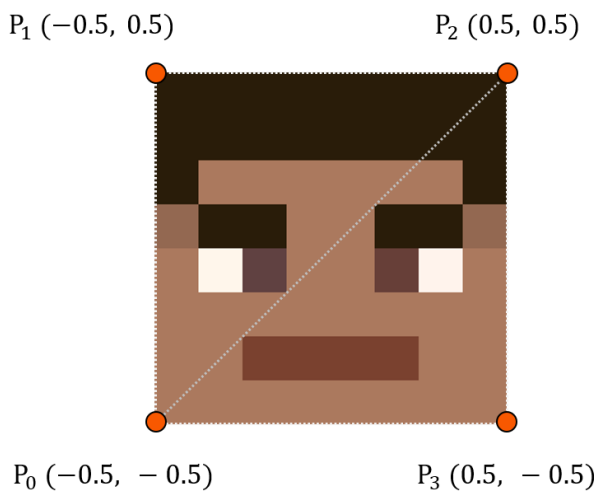


Heesung Oh
Copyright All rights reserved

마인크래프트 스킨에서 얼굴 부분의 UV 좌표



각 정점에 설정한 UV 값



정점 버퍼

인덱스	좌표	UV좌표
0	$(-0.5,\ -0.5)$	$(0.125,\ 0.75)$
1	$(-0.5,\ 0.5)$	$(0.125,\ 0.875)$
2	$(0.5,\ 0.5)$	$(0.25,\ 0.75)$
3	$(0.5,\ -0.5)$	$(0.25,\ 0.875)$

인덱스 버퍼

인덱스	삼각형	정점인덱스
0	0	0
1	0	1
2	0	2
3	1	0
4	1	2
5	1	3

8-4 텍스처 매핑의 구현

Heesung Oh
Copyright All rights reserved

```
Vector2 targetUV = tv[0].UV * oneMinusST
                 + tv[1].UV * s
                 + tv[2].UV * t;
```

8.5 정리

Heesung Oh
Copyright All rights reserved

- 아핀 결합을 사용한 삼각형 표현
- 삼각형
- 무게중심 좌표
- 텍스처 좌표

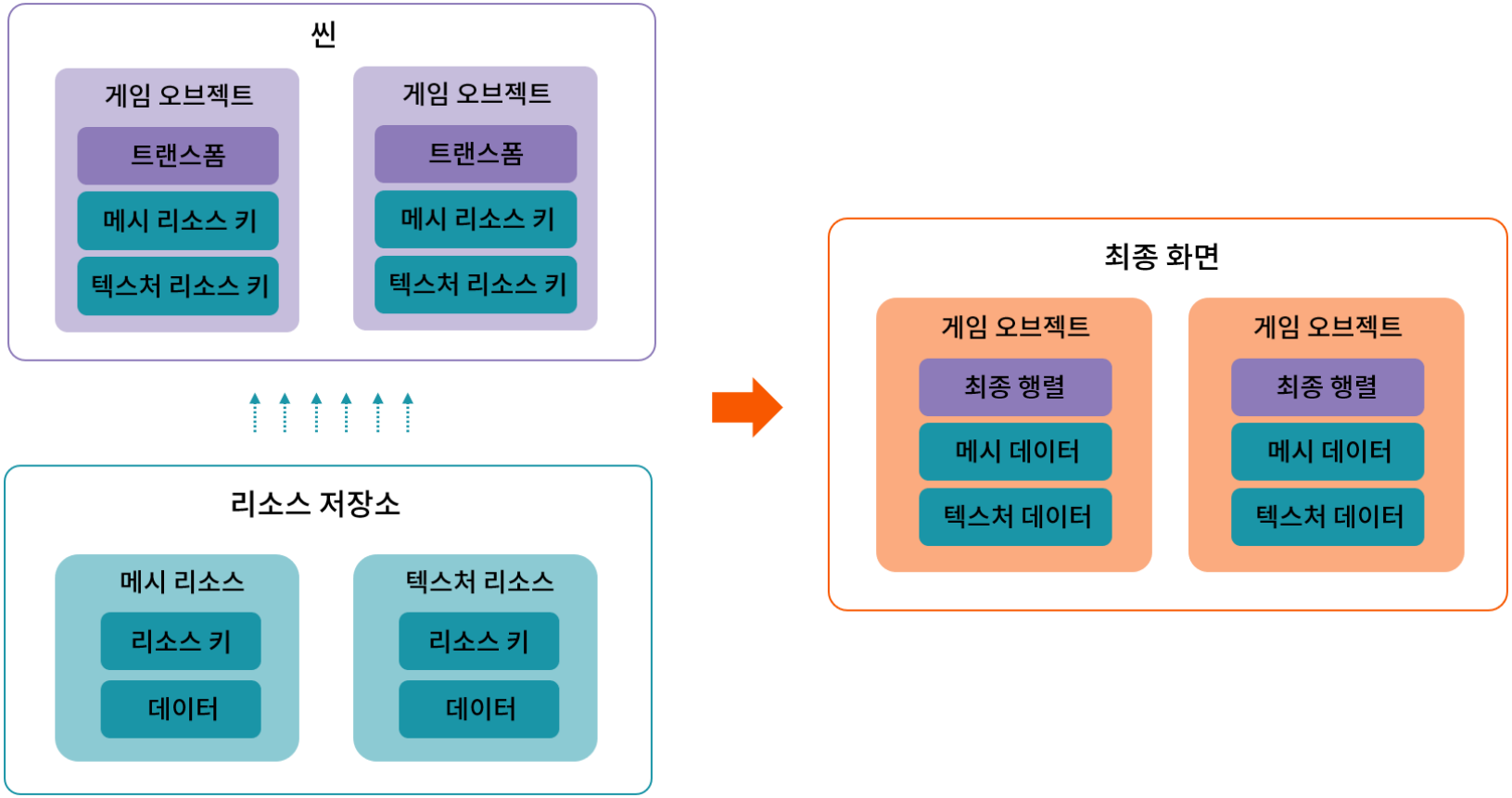
Copyright All rights reserved

9. 게임 엔진

콘텐츠를 만드는 기술

Heesung Oh
Copyright All rights reserved

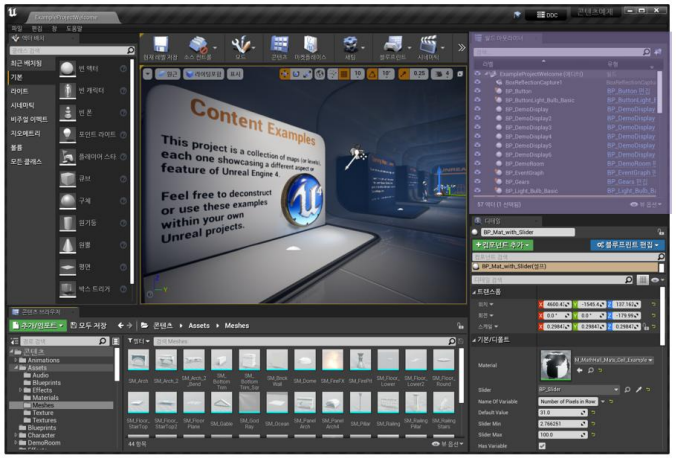
- 씬과 리소스로부터 만들어지는 최종 화면의 렌더링



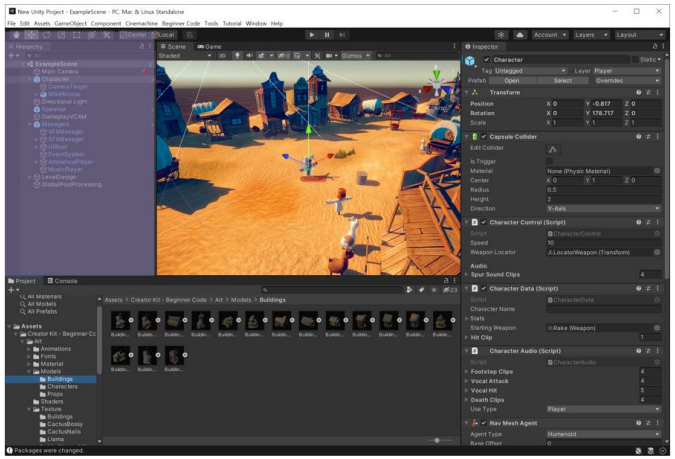
Heesung Oh
Copyright All rights reserved

- 엔진의 Scene 구성

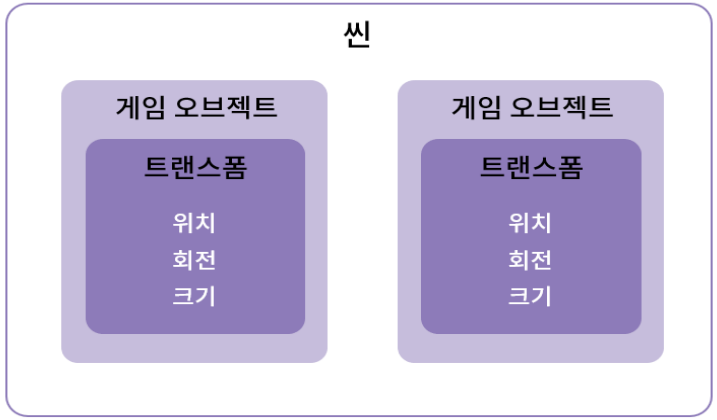
(a) 언리얼 엔진



(b) 유니티 엔진



- 씬, 게임 오브젝트, 트랜스폼의 관계



9.1.2 모델링 행렬의 설계

Heesung Oh
Copyright All rights reserved

- **트랜스폼**
  - **크기**

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

  - **회전**

$$R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

  - **이동**

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

  - **왼손 좌표계**

$$R = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

9.1.2 모델링 행렬의 설계

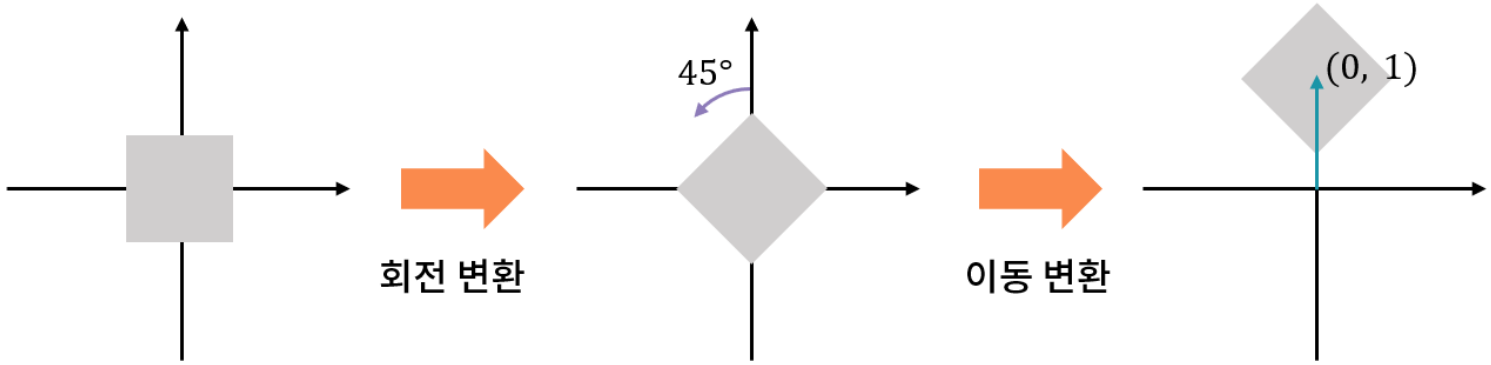
Heesung Oh
Copyright All rights reserved

- S·R·T
- S·T·R
- T·S·R
- T·R·S
- R·T·S
- R·S·T

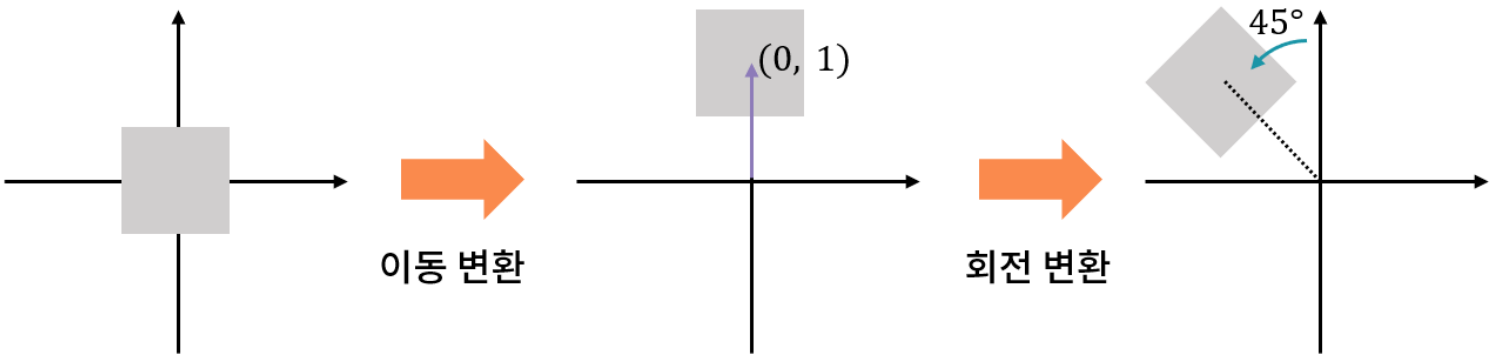
Heesung Oh
Copyright All rights reserved

- 이동 변환과 회전 변환의 적용 순서에 따른 차이 비교

(a)



(b)

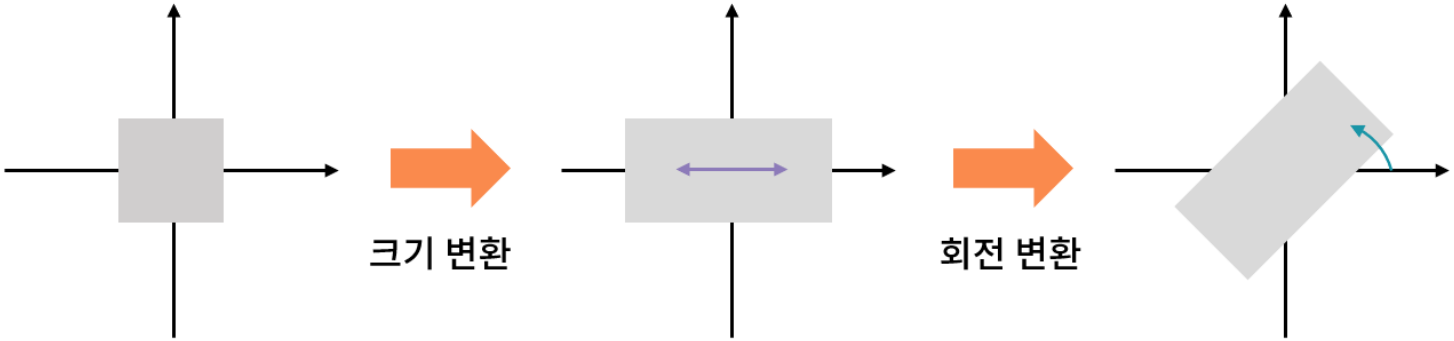


9.1.2 모델링 행렬의 설계

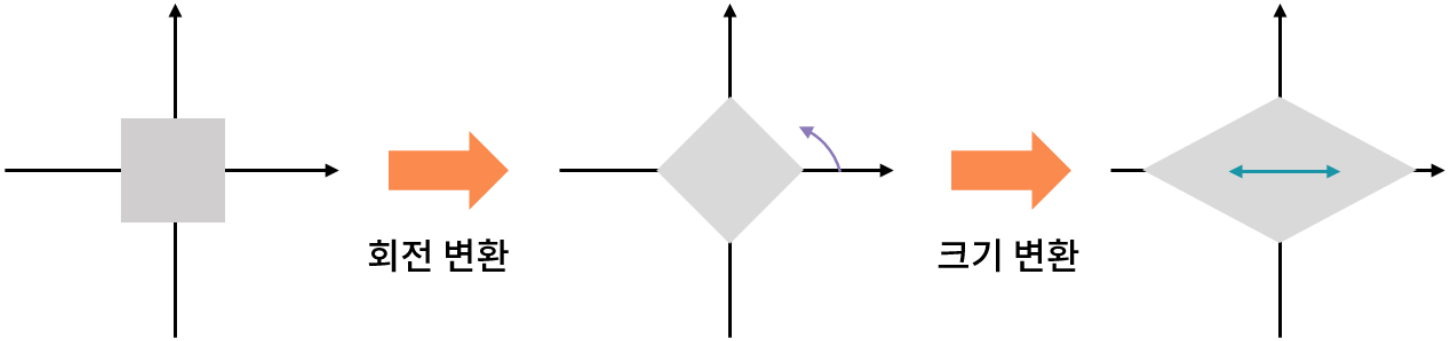
Heesung Oh
Copyright All rights reserved

- 회전 변환과 크기 변환의 적용 순서에 따른 차이 비교

(a)



(b)



9.1.2 모델링 행렬의 설계

Heesung Oh
Copyright All rights reserved

- **오른손 좌표계**

$$M_{r-hand} = T \cdot R \cdot S$$
$$= \begin{bmatrix} \cos\theta \cdot s_x & -sin\theta \cdot s_y & t_x \\ \sin\theta \cdot s_x & \cos\theta \cdot s_y & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- **왼손 좌표계**

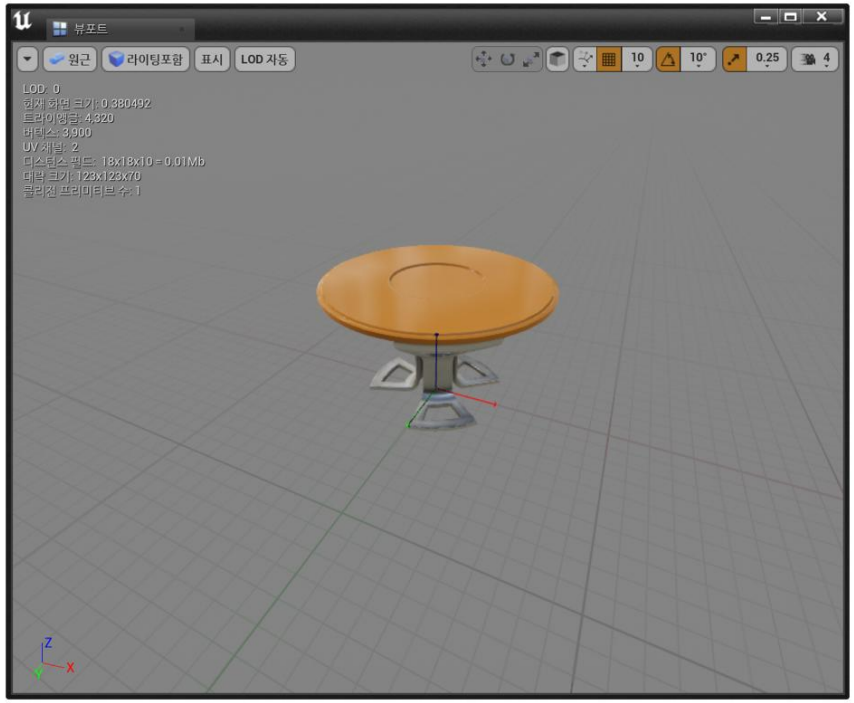
$$M_{l-hand} = \begin{bmatrix} \cos\theta \cdot s_x & \sin\theta \cdot s_y & 0 \\ -\sin\theta \cdot s_x & \cos\theta \cdot s_y & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

Heesung Oh
Copyright All rights reserved

- 게임 엔진의 월드 공간과 로컬 공간

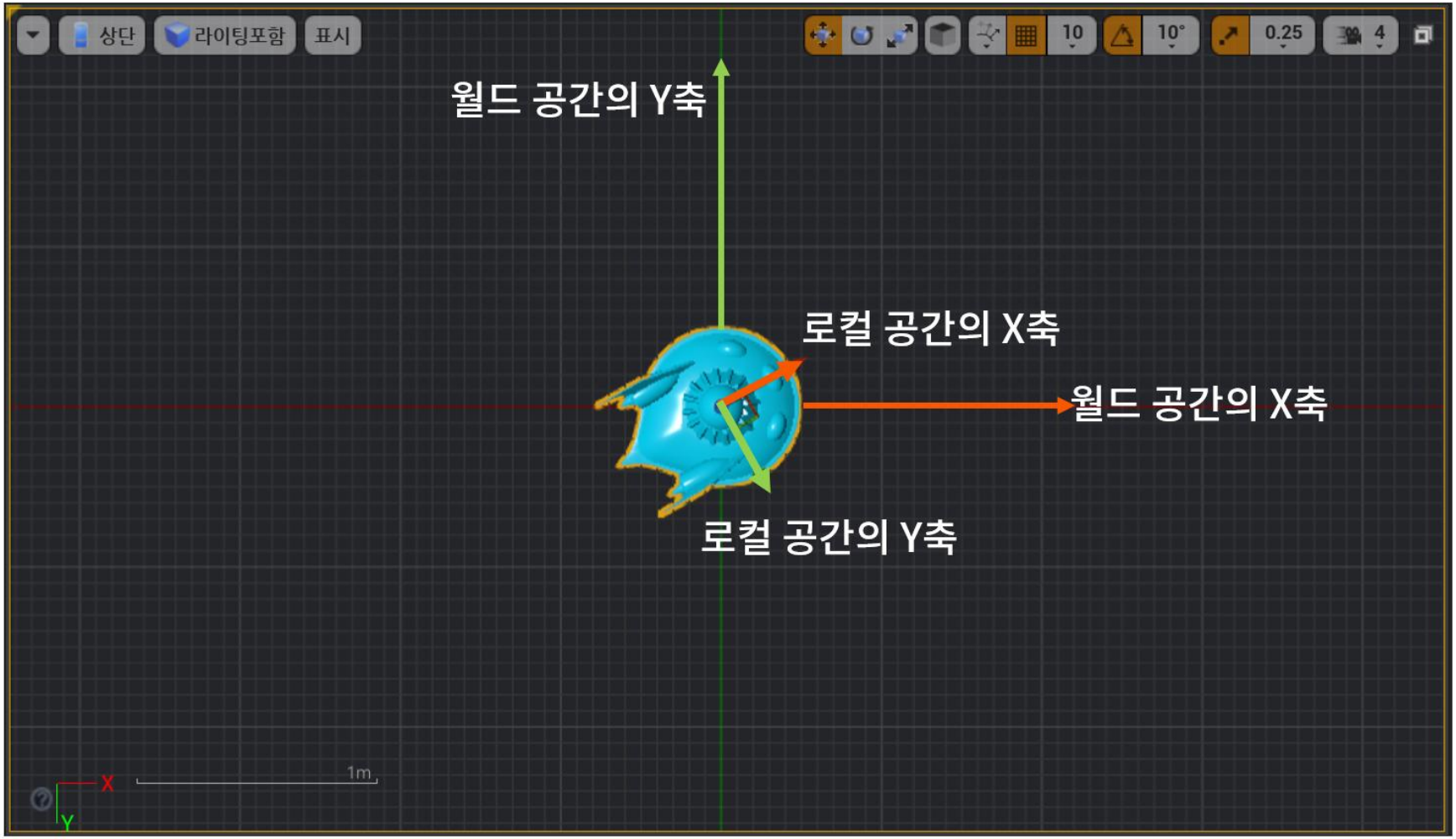
(a)

(b)



Heesung Oh
Copyright All rights reserved

- 월드 공간의 축과 로컬 공간의 축



9.1.3 로컬 공간과 로컬 축

Heesung Oh
Copyright All rights reserved

- **표준 기저 벡터 e_1**

$$\vec{r} = (\cos\theta, \sin\theta)$$

- **표준 기저 벡터 e_2**

$$\vec{u} = (-\sin\theta, \cos\theta)$$

- **Rotation Matrix**

$$R = \begin{bmatrix} r_x & u_x & 0 \\ r_y & u_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Local 공간 Transform**

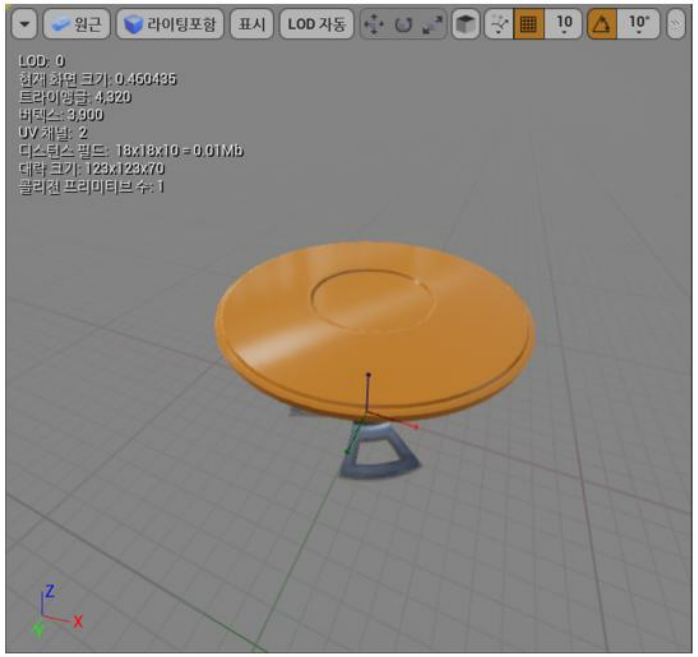
$$M_{Local} = \begin{bmatrix} r_x \cdot s_x & u_x \cdot s_y & t_x \\ r_x \cdot s_y & u_y \cdot s_y & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

9.1.4 리소스 관리

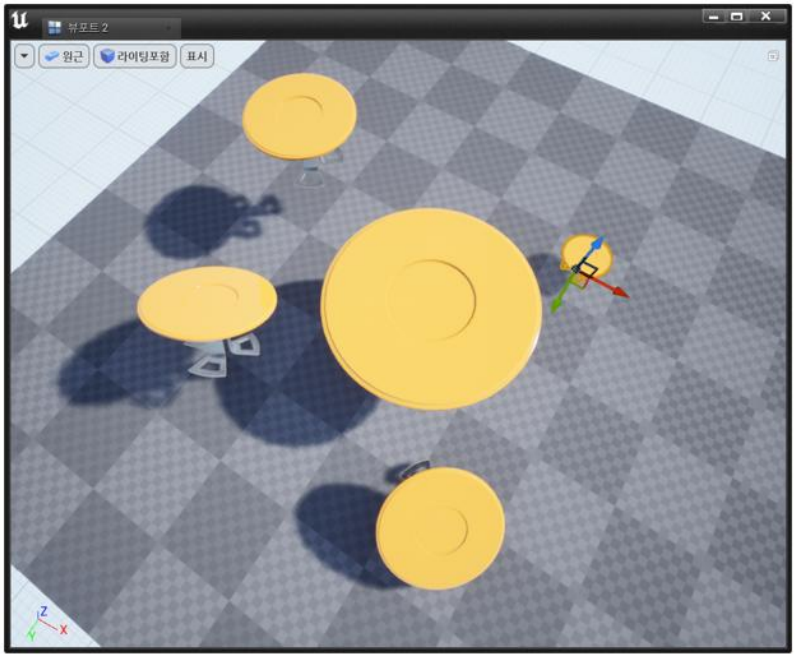
Heesung Oh
Copyright All rights reserved

- 동일한 리소스를 사용하는 다수의 게임 오브젝트를 화면에 배치한 결과

로컬 공간

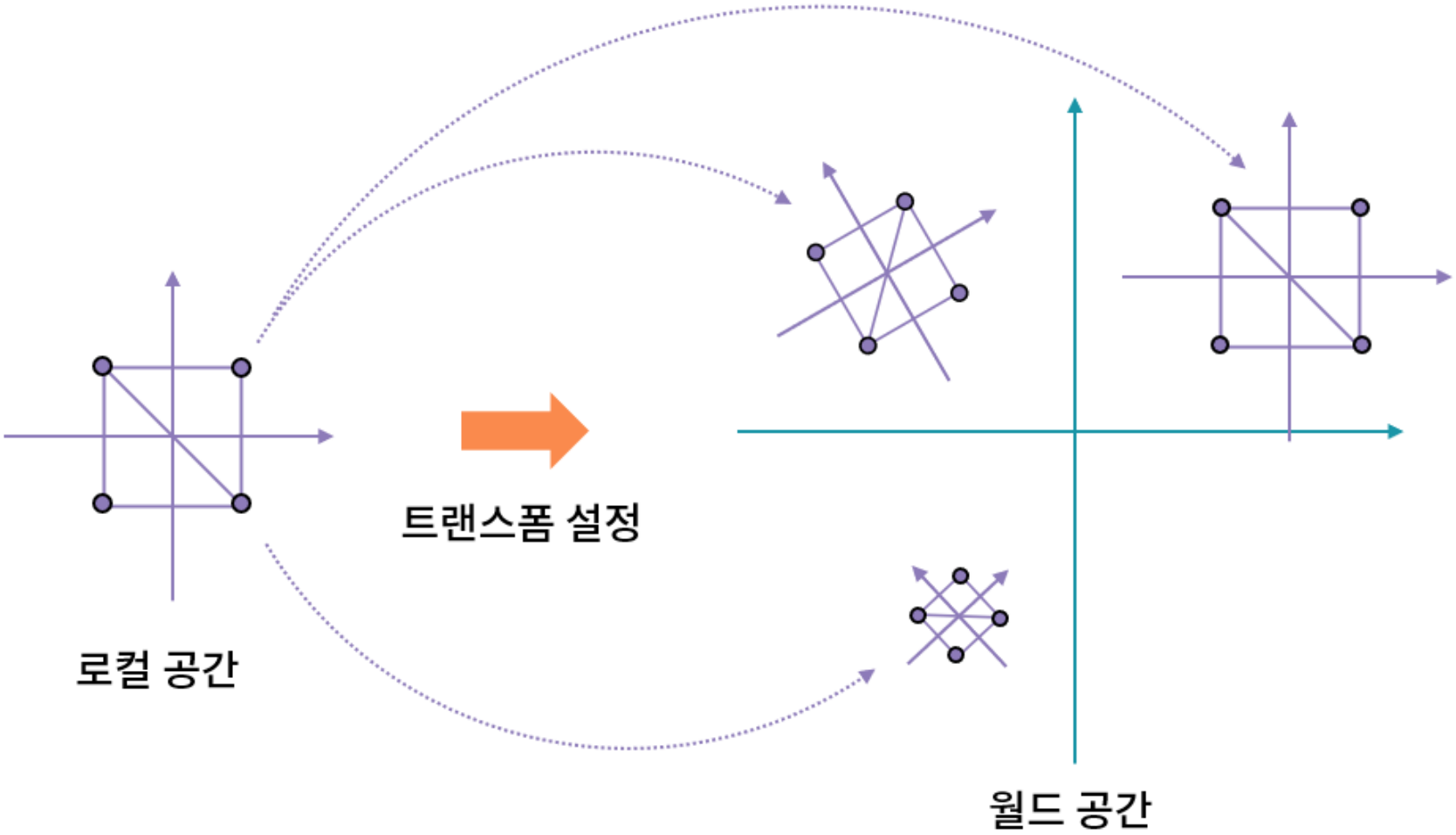


월드 공간



Heesung Oh
Copyright All rights reserved

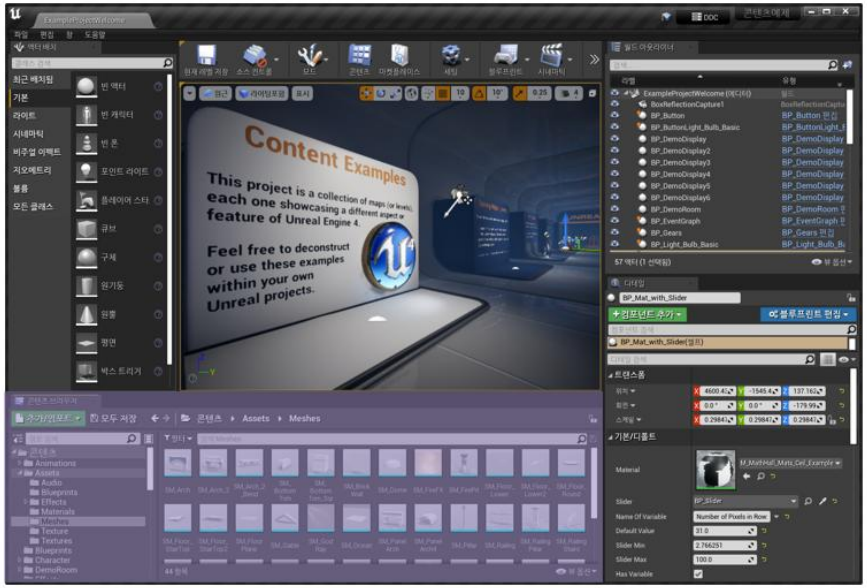
- 게임 콘텐츠의 제작 과정



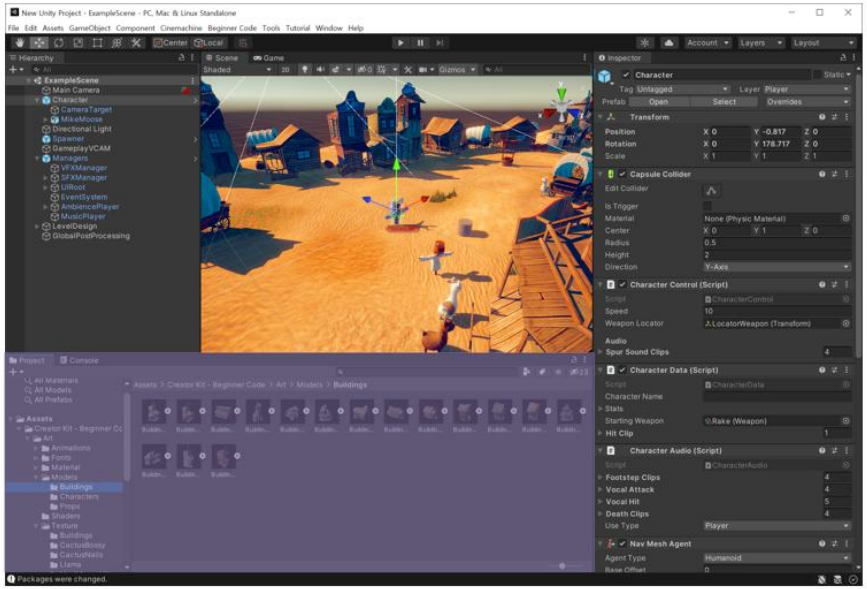
Heesung Oh
Copyright All rights reserved

- 게임 엔진이 제공하는 리소스 저장소 영역

(a) 언리얼 엔진



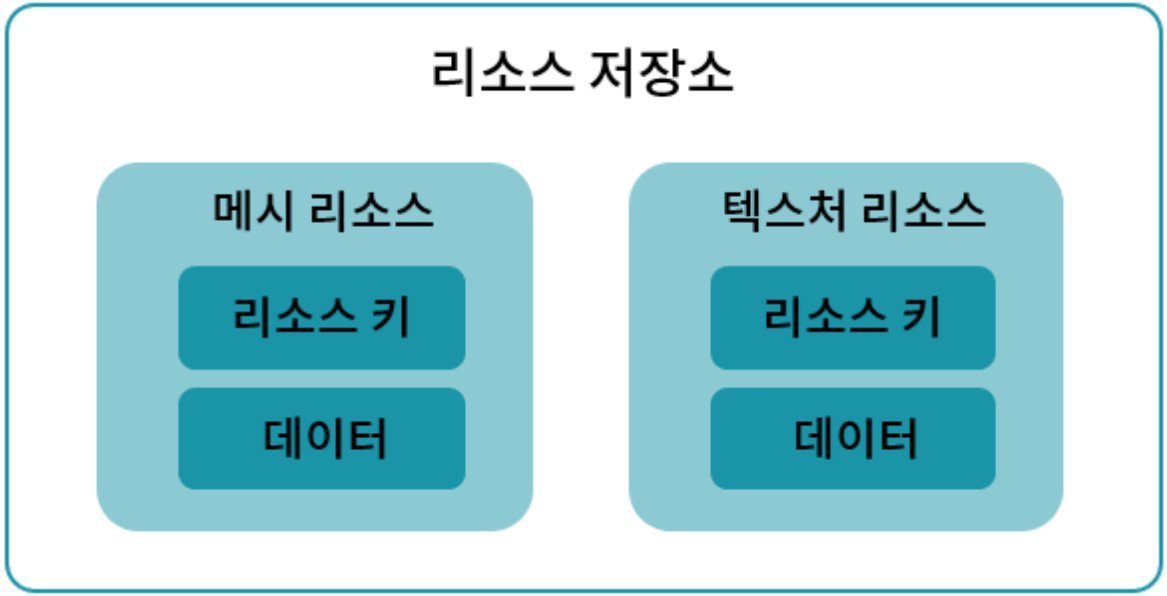
(b) 유니티 엔진



9.1.4 리소스 관리

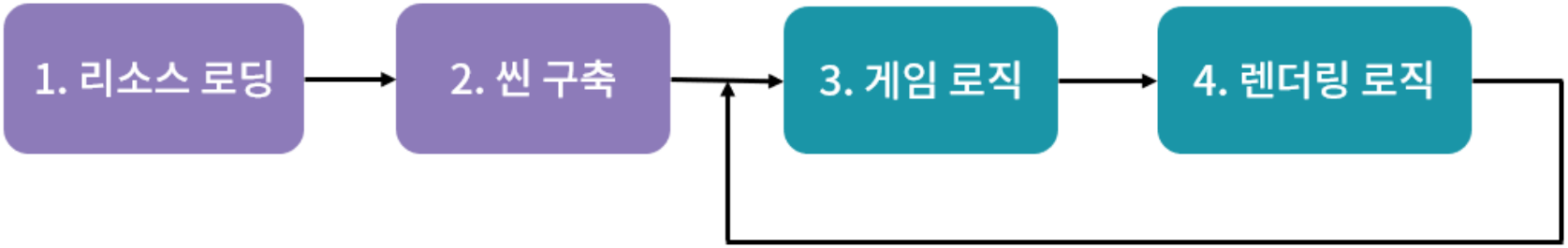
Heesung Oh
Copyright All rights reserved

- Scene
  - 게임 월드 공간
  - Level
- 리소스
  - 데이터
  - 에셋(Asset)
    - 관리 형태
- 리소스 관리
  - {키 + 데이터}



Heesung Oh
Copyright All rights reserved

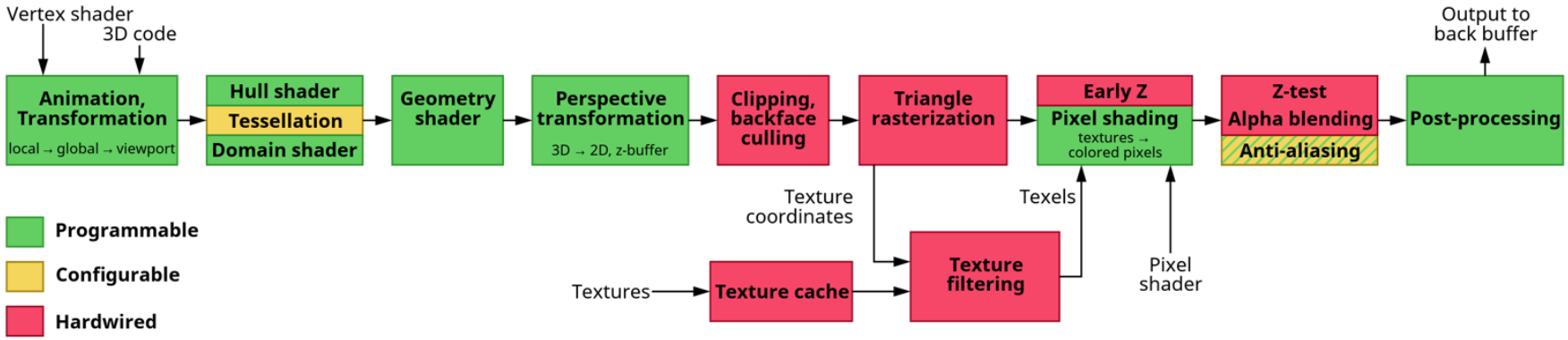
9.2.1 게임 엔진의 진행 과정



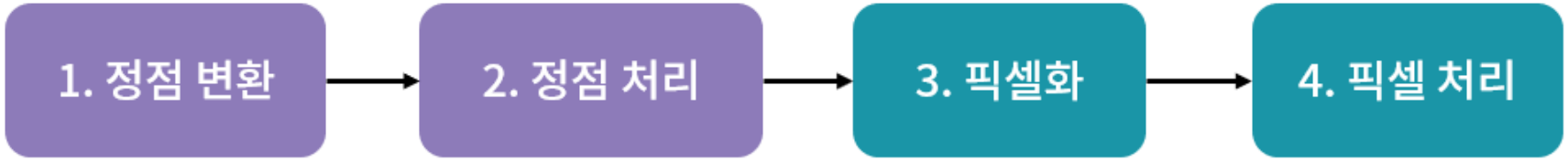
- 리소스 로딩: LoadResources
- 씬 구축: LoadScene2D
- 게임 로직: Update2D
- 렌더링: Render2D

Heesung Oh
Copyright All rights reserved

- GPU에서 수행되는 렌더링 파이프라인



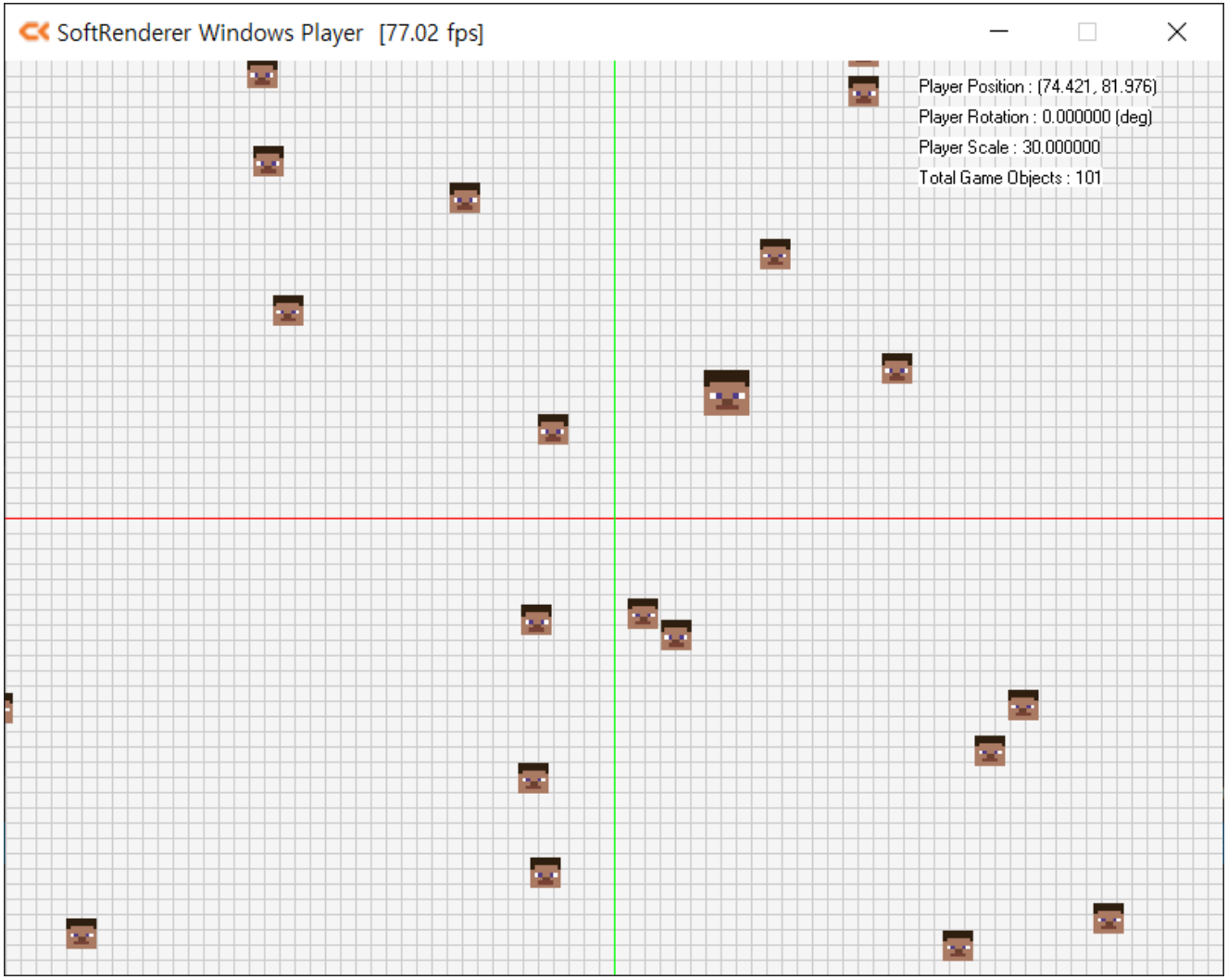
- CK 소프트렌더러의 렌더링 파이프라인



9.2.2 렌더링 파이프라인

Heesung Oh
Copyright All rights reserved

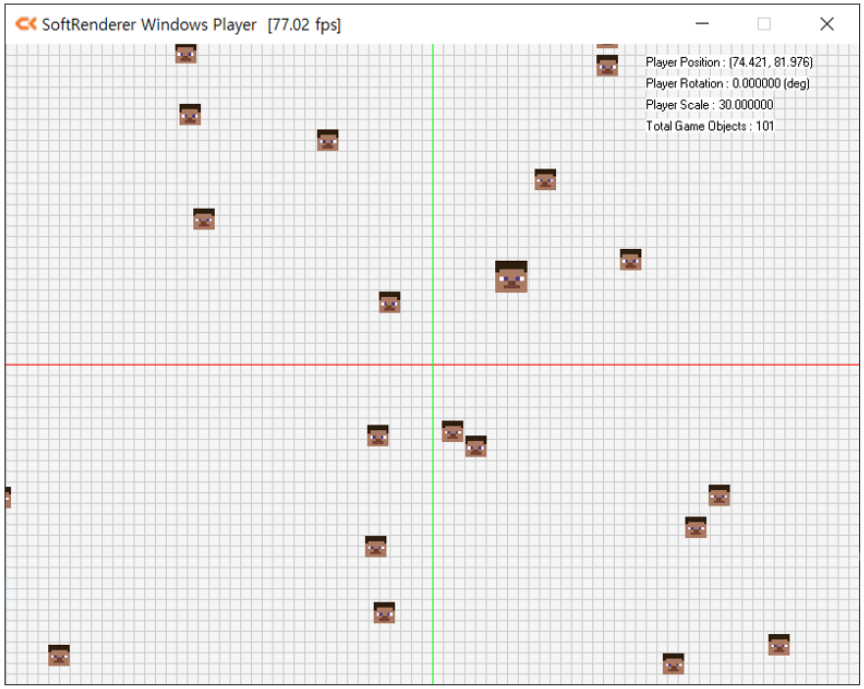
- 월드 공간을 중심으로 각 게임 오브젝트를 그린 결과 화면



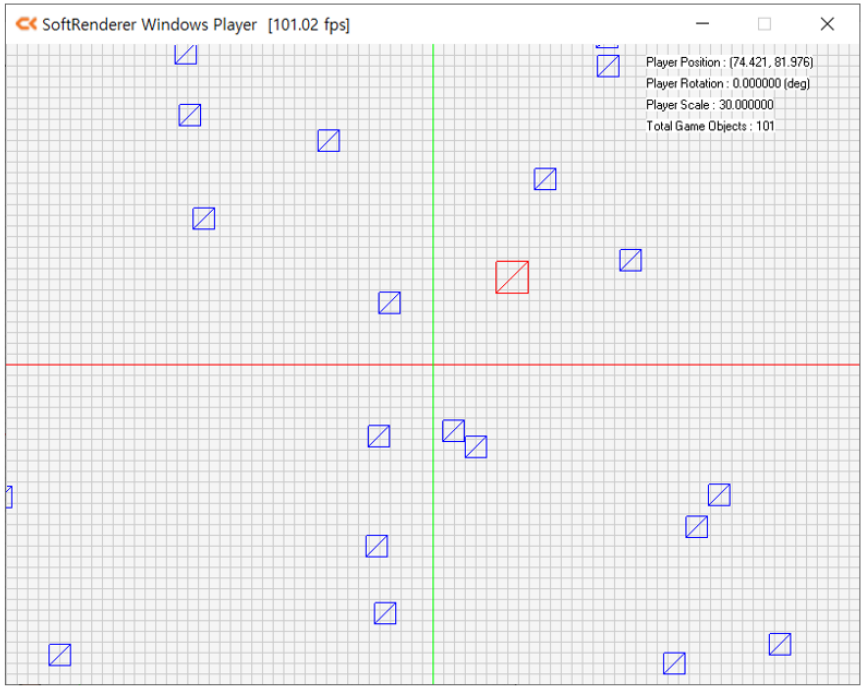
Heesung Oh
Copyright All rights reserved

- 일반 렌더링 모드와 와이어프레임 렌더링 모드의 차이

(a) 일반 렌더링 모드

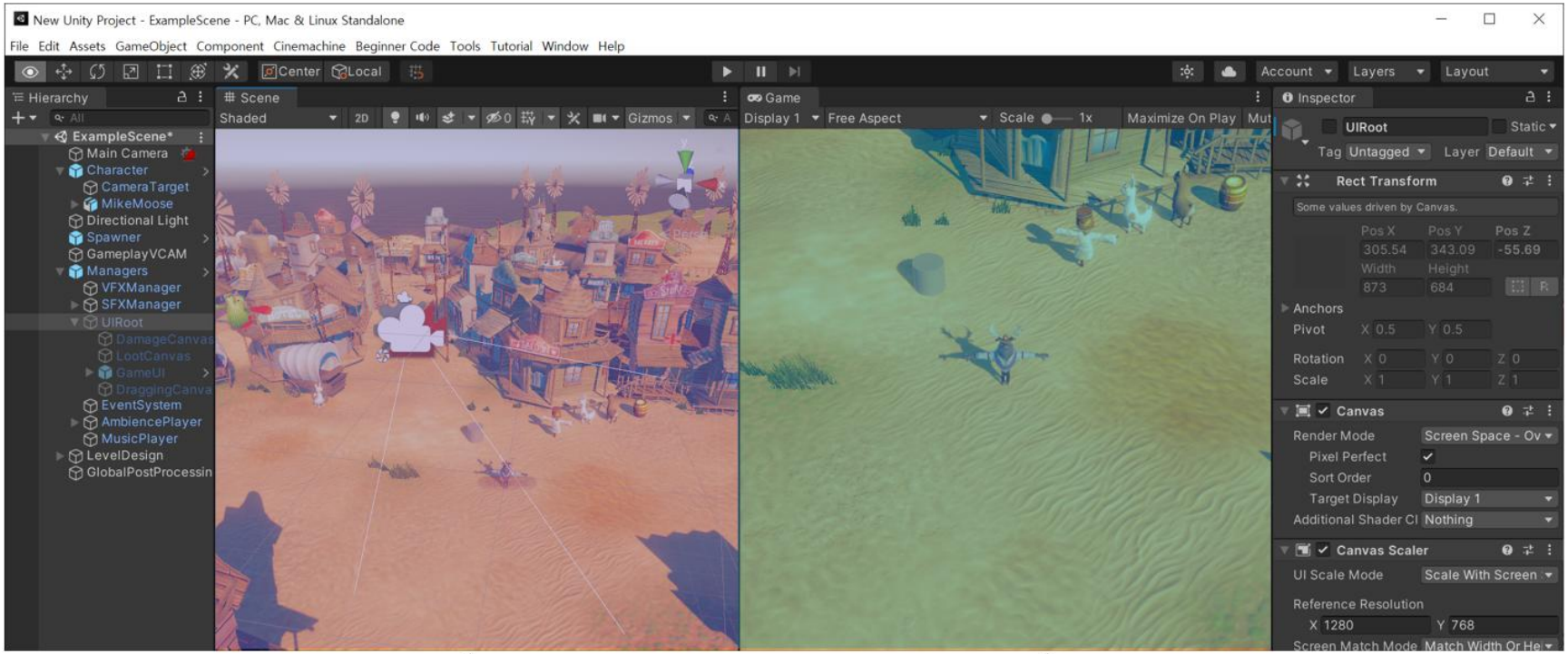


(b) 와이어프레임 렌더링



Heesung Oh
Copyright All rights reserved

- 개발자가 보는 게임 공간과 게이머가 보는 게임 공간

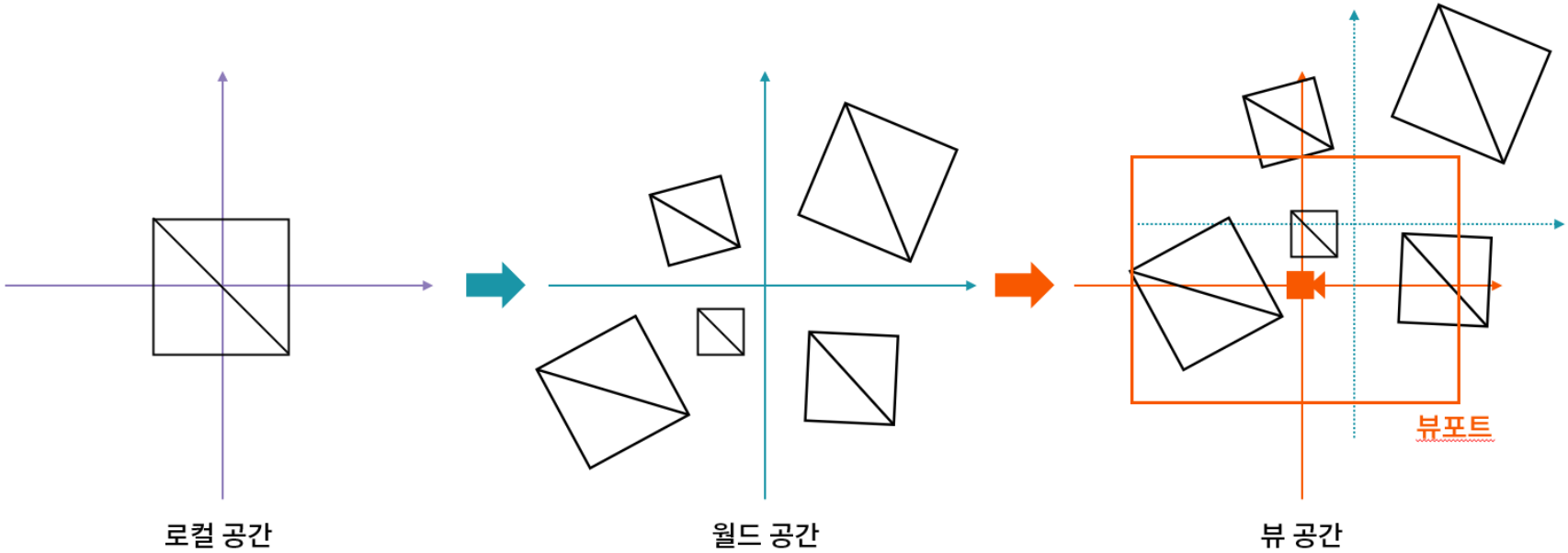


개발자가 보는 월드 공간

게이머가 보는 월드 공간

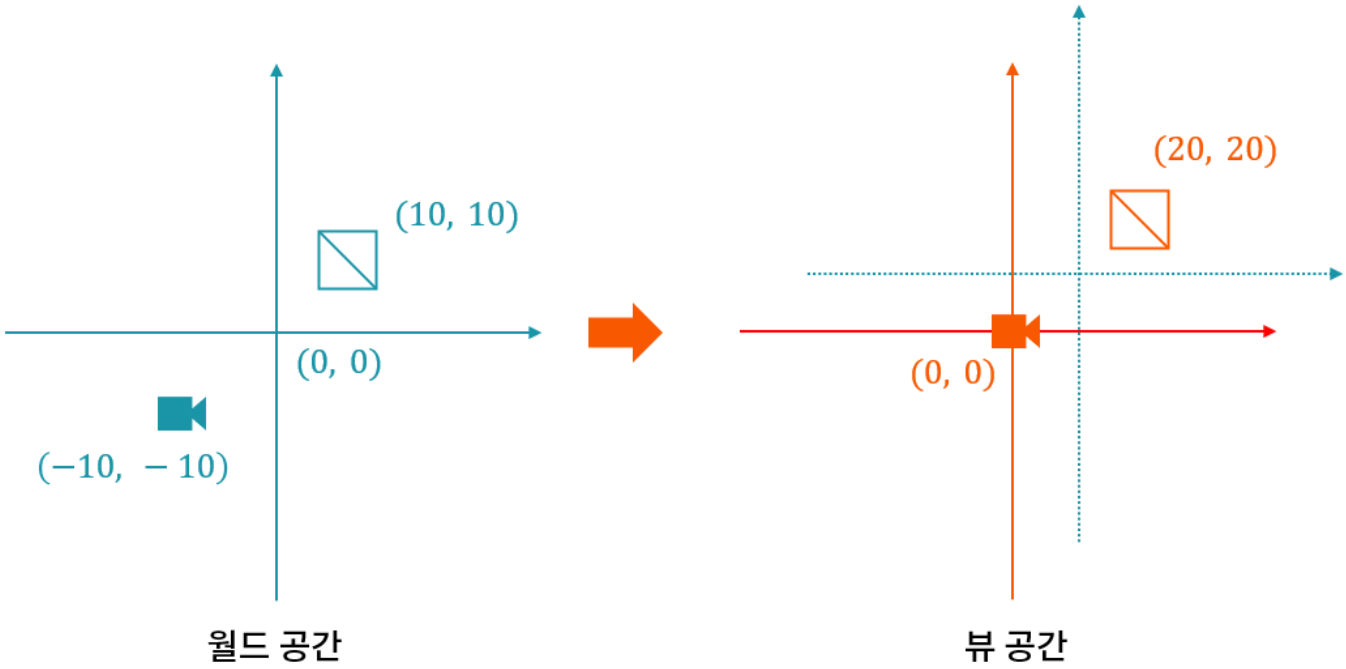
Heesung Oh
Copyright All rights reserved

- **로컬 공간, 월드 공간, 뷰 공간의 개요**



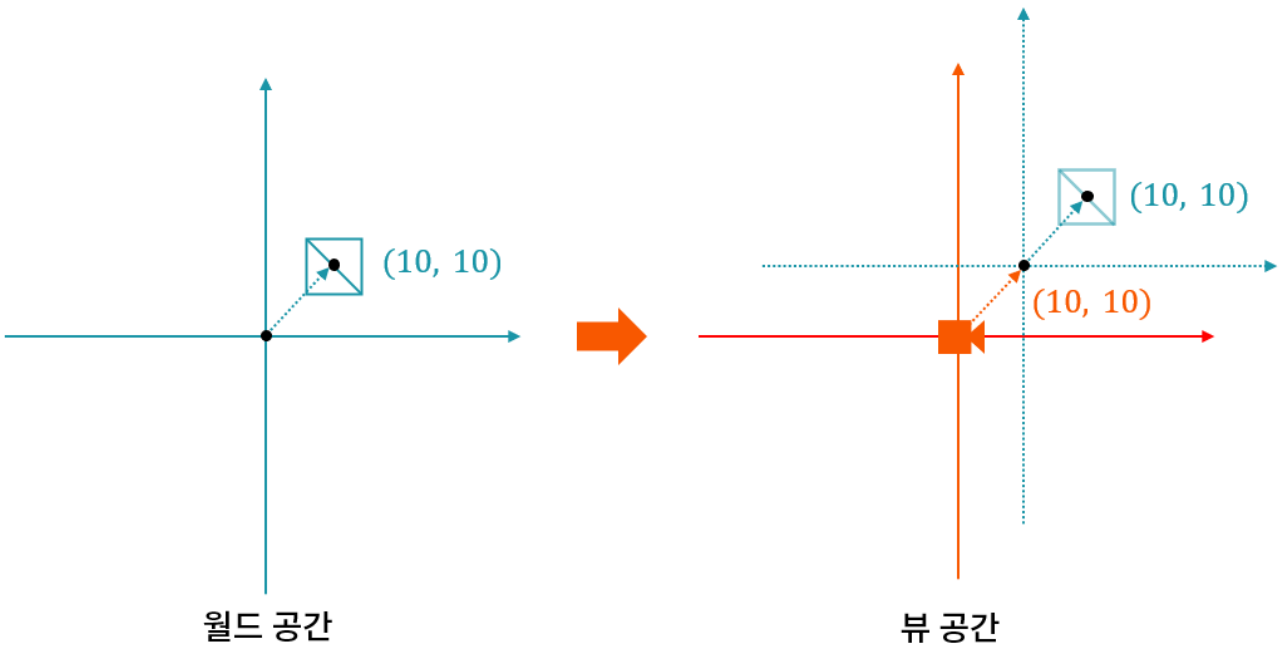
Heesung Oh
Copyright All rights reserved

- 카메라를 중심으로 해석한 게임 오브젝트의 최종 위치



Heesung Oh
Copyright All rights reserved

- **카메라를 중심으로 해석한 게임 오브젝트의 최종 위치**



$$V = \begin{bmatrix} 1 & 1 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

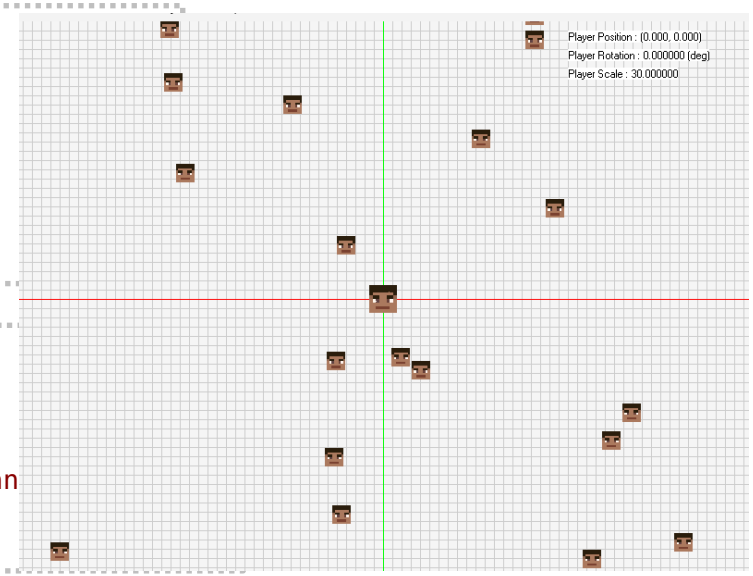
$$V_{view} = V \cdot M \cdot v_{local}$$

실습: 9-1 게임 월드의 탐험

Heesung Oh
Copyright All rights reserved

```
void SoftRenderer::LoadScene2D()
{
    // 최초 씬 로딩에서 사용하는 모듈 내 주요 레퍼런스
    auto& g = Get2DGameEngine();

    // 플레이어의 생성과 설정
    constexpr float playerScale = 30.f;
    GameObject& goPlayer = g.CreateNewGameObject(PlayerGo);
```




```
// 게임 로직을 담당하는 함수
void SoftRenderer::Update2D(float InDeltaSeconds)
{
    // 플레이어를 따라다니는 카메라의 트랜스폼
    TransformComponent& cameraTransform = g.GetMainCamera().GetTran
    Vector2 playerPos = transform.GetPosition();
    Vector2 cameraPos = cameraTransform.GetPosition();
```

```
// 렌더링 로직을 담당하는 함수
void SoftRenderer::Render2D()
{
    // 렌더링 로직의 로컬 변수
    size_t totalObjectCount = g.GetScene().size();
    Matrix3x3 viewMatrix = g.GetMainCamera().GetViewMatrix();

    // 씬을 구성하는 모든 게임 오브젝트의 순회
    for (auto it = g.SceneBegin(); it != g.SceneEnd(); ++it)
    {

        // 렌더링에 필요한 게임 오브젝트의 주요 레퍼런스를 얻기
        const Mesh& mesh = g.GetMesh(gameObject.GetMeshKey());
        const TransformComponent& transform = gameObject.GetTransform();
        Matrix3x3 finalMatrix = viewMatrix * transform.GetModelingMatrix();

        // 게임 오브젝트의 렌더링 수행
        DrawMesh2D(mesh, finalMatrix, gameObject.GetColor());
```

9.4 정리

Heesung Oh
Copyright All rights reserved

- 게임엔진
  - 씬
    - 게임 오브젝트
      - 트랜스폼
  - 리소스 저장소
    - 메시, 텍스처
  - 화면 구성
    - 게임 오브젝트 + 렌더링 데이터
  - 렌더링 파이프라인

- 회전변환, 크기 변환, 이동 변환의 적용 순서에 대한 차이 비교
- 월드 공간, 로컬 공간 차이
- 오브젝트의 화면 배치